Erkennung und semantische Klassifikation von Formularfeldern mittels Computer Vision und LLMs

Bachelorarbeit

an der

FH Aachen

Fachbereich
Elektrotechnik und Informationstechnik

Vorgelegt von
Manuel Ben João Salomon
Matr.-Nr.: 3232880
aus Aachen

Erstprüfer: Prof. Dr. rer. nat. Heinrich Faßbender
Korreferent: M.Eng. Torben Hensgens

21. April 2026

Kurzreferat

Die vorliegende Bachelorarbeit geschrieben von Ben Salomon, befasst sich mit der automatischen Erkennung und semantischen Klassifikation von Formularfeldern in PDF-Dokumenten. Ziel ist die Entwicklung eines Systems, das den manuellen Aufwand bei der Erstellung interaktiver PDF-Formulare reduziert.
Die zentrale Forschungsfrage lautet, wie Computer Vision Algorithmen zur automatischen Erkennung und Klassifizierung von Eingabefeldern in PDF-Formularen eingesetzt werden können und welche technischen Herausforderungen dabei bestehen.
Zur Beantwortung dieser Frage wurde ein Prototyp entwickelt, der klassische Bildverarbeitungsmethoden mit modernen Sprachmodellen kombiniert. Die Verarbeitungspipeline nutzt OpenCV für die Felderkennung mittels Kantendetektion und Konturanalyse, PDFMiner für die strukturierte Textextraktion sowie Tesseract OCR für die optische Zeichenerkennung. Für die semantische Zuordnung von Labels zu Feldern wird ein räumlicher Analysealgorithmus eingesetzt, der optional durch Large Language Models ergänzt werden kann. Das System unterstützt dabei mehrere LLM-Anbieter wie OpenAI GPT-4, Google Gemini und lokale Modelle über Ollama.
Das entwickelte Web-Interface auf Basis von Django ermöglicht die Verwaltung von Templates und Datasets sowie die automatisierte PDF-Generierung. Die Ergebnisse zeigen, dass der gewählte Ansatz Formularfelder erkennen und klassifizieren kann, wobei die Kombination aus regelbasierter Bildverarbeitung und optionaler KI-Unterstützung einen praktikablen Kompromiss zwischen Genauigkeit und Flexibilität bietet.

Inhaltsverzeichnis

Kurzreferat **II**

Abkürzungsverzeichnis **VI**

Abbildungsverzeichnis **VII**

Tabellenverzeichnis **IX**

1 Einleitung **1**
1.1 Problemstellung und Motivation ... 1
1.2 Forschungsfrage ... 1
1.3 Aufbau der Arbeit ... 2

2 Systemarchitektur **3**
2.1 Überblick der Komponenten & Technologie-Stack ... 3
2.2 Datenbankdesign ... 4
2.2.1 Entity-Relationship-Modell ... 4
2.2.2 Datenfluss-Architektur ... 4
2.2.3 Schnittstellendesign ... 5
2.2.4 Konfigurationsmanagement ... 5

3 Computer Vision Algorithmus **7**
3.1 Überblick der Verarbeitungspipeline ... 7
3.2 PDF-Verarbeitung und Textextraktion ... 8
3.2.1 PDF zu Bild Konvertierung ... 8
3.2.2 Strukturierte Textextraktion mit PDFMiner ... 9
3.2.3 Koordinatensystem-Transformation ... 10
3.3 Bildvorverarbeitung und Felderkennung ... 10
3.3.1 Preprocessing-Pipeline ... 10
3.3.2 Konturdetektion und Rechteckerkennung ... 11
3.4 Intelligente Label-Zuordnung ... 13
3.4.1 Räumliche Beziehungsanalyse ... 13
3.4.2 Label-Genauigkeitsbewertung ... 13
3.4.3 Greedy Label-Feld-Zuordnung ... 13
3.5 Ausgabe-Generierung und Visualisierung ... 13
3.5.1 Template-Format-Konvertierung ... 15
3.5.2 Visuelle Ergebnisdarstellung ... 15

4 Semantische Label-Feld-Zuordnung durch KI-Modelle **17**
4.1 Motivation: Wo der geometrische Ansatz scheitert ... 17
4.1.1 Beispiel 1: Auftragsformular ... 17

4.1.2 Beispiel 2: Bewerbungsformular ... 19
4.1.3 Ableitung der Fehlermuster ... 21
4.2 Grundlagen: LLMs für strukturierte Dokumentenanalyse ... 21
4.2.1 Semantische Repräsentation von Formularstrukturen ... 22
4.2.2 Kontextfenster und Prompt-basierte Steuerung ... 22
4.2.3 Chain-of-Thought und Reasoning-Modelle ... 22
4.3 Systemarchitektur der LLM-Integration ... 22
4.3.1 Einordnung in den Verarbeitungsfluss ... 23
4.3.2 Der LabelFieldAgent als Abstraktionsschicht ... 23
4.3.3 Fehlertoleranz und Retry-Mechanismus ... 23
4.4 Prompt Engineering für Formularanalyse ... 24
4.4.1 Struktur des Prompts ... 24
4.4.2 Designentscheidungen im Prompt ... 25
4.4.3 Token-Verbrauch und Skalierungsverhalten ... 25
4.5 Vergleichende Analyse: Traditionell vs. LLM ... 26
4.5.1 Kostenbetrachtung im Praxiskontext ... 26
4.5.2 Bewertung der Genauigkeitsunterschiede ... 27
4.6 Robustheit und Grenzen des LLM-Ansatzes ... 27
4.6.1 Nicht-Determinismus ... 27
4.6.2 API-Abhängigkeit und Datenschutz ... 27
4.6.3 Qualität der JSON-Ausgabe ... 28
4.6.4 Skalierbarkeit bei mehrseitigen Dokumenten ... 28

5 Web-Interface und Management-System 29
5.1 Architektur und technische Grundlagen ... 29
5.1.1 Warum Django? ... 29
5.1.2 MVT-Architekturmuster ... 30
5.1.3 Navigation und Oberflächenstruktur ... 30
5.2 Template-Verwaltung ... 30
5.2.1 Template-Übersicht ... 30
5.2.2 Automatisierte Template-Generierung ... 31
5.2.3 Template-Detailansicht ... 33
5.2.4 Template-Bearbeitung ... 34
5.3 PDF-Generierung ... 34
5.3.1 Eingabeformular ... 34
5.3.2 Technische Verarbeitung bei der Generierung ... 34
5.4 Dataset-Management ... 35
5.4.1 Dataset-Erstellung ... 35
5.4.2 Dataset-Übersicht und Anwendung ... 36
5.5 Generierungshistorie ... 37
5.6 Beispieldurchlauf: Von der Quelldatei zur befüllten PDF ... 38
5.6.1 Schritt 1: Template automatisch generieren ... 38
5.6.2 Schritt 2: Ergebnis prüfen und ggf. korrigieren ... 38
5.6.3 Schritt 3: Erste PDF generieren ... 38
5.6.4 Schritt 4: Dataset aus der Generierung erstellen ... 39
5.6.5 Schritt 5: Dataset für weitere Generierungen nutzen ... 39

6 Fazit und Ausblick 40
6.1 Beantwortung der Forschungsfrage ... 40

6.2 Stärken und Schwächen des Ansatzes 41
6.2.1 Stärken 41
6.2.2 Schwächen 41
6.3 Technische Designentscheidungen und ihre Bewertung 42
6.4 Potenzielle Verbesserungen und zukünftige Erweiterungen 42

Quellenverzeichnis **44**

Anhang **45**

Erklärung **46**

Abkürzungsverzeichnis

API	Application Programming Interface
CRUD	Create, Read, Update, Delete
HTML	Hypertext Markup Language
HTTP	Hypertext Transfer Protocol
JSON	JavaScript Object Notation
UUID	Universally Unique Identifier
MVT	Model-View-Template
CV	Computer Vision
DPI	Dots Per Inch
PIL	Python Imaging Library
OCR	Optical Character Recognition
RGB	Red, Green, Blue
PDF	Portable Document Format
ITU	International Telecommunication Union
KI	Künstliche Intelligenz
LLM	Large Language Model
GPT	Generative Pre-trained Transformer
ORM	Object-Relational Mapping

Abbildungsverzeichnis

2.1 Entity-Relationship-Diagramm der Datenbank 4

3.1 Vollständiger Datenfluss des Computer Vision Algorithmus mit farblicher Kodierung der Verarbeitungsebenen 7
3.2 Beispiel PDF Formular (Rechnung, wurde aus Platzgründen unten etwas gekürzt) 8
3.3 Graustufenkonvertierung 12
3.4 Gaußsche Glättung 12
3.5 Canny-Kantendetektion 12
3.6 Visualisierung der Preprocessing-Schritte zur Extraktion definierter Formularfeld-Kanten. 12
3.7 Alle erkannten 4-Punkt-Polygone 12
3.8 Nach Filterung ($w \geq 40$, $h \geq 20$) 12
3.9 Die Größeneinschränkung eliminiert erfolgreich Störartefakte und Buchstaben, die zufällig eine viereckige Struktur aufweisen. 12
3.10 Jedes Eingabefeld (grün) wird seinem semantisch nächsten Label-Text (blau) zugeordnet. 14
3.11 Finales Output: Erfolgreich extrahierte und benannte Formularfelder (grün), bereit zur Verarbeitung durch die nachfolgende Template-Engine. 16

4.1 Komplexes Auftragsformular mit mehrspaltigen Abschnitten, geteilten Spaltenlabels und einem unkonventionell positionierten Label (← *R.-Datum*) rechts vom Feld 18
4.2 Traditioneller Algorithmus auf dem Auftragsformular: Das Feld für R.-Datum erhält `None`, da sein Label rechts davon steht. Das darunter liegende Feld wird fälschlicherweise mit *Kundennummer:* beschriftet. 18
4.3 LLM-Ansatz (Gemini 2.5 Flash) auf demselben Formular: Alle 9 Felder werden korrekt zugeordnet, inklusive *R.-Datum* und der Tabellenfelder der zweiten Zeile. 18
4.4 Bewerbungsformular mit drei verschiedenen Layoutfallen: Vorname und Nachname als Linienfelder (Abschnitt 1), drei eng beieinanderliegende Checkboxen mit Labels (Abschnitt 3) und ein Unterschriftsfeld mit Label unterhalb statt oberhalb. 19
4.5 Traditioneller Algorithmus auf dem Bewerbungsformular: Vorname und Nachname werden nicht erkannt (keine Kontur), die Checkbox-Labels sind um eine Position verschoben zugeordnet. 20
4.6 LLM-Ansatz (Gemini 2.5 Flash) auf demselben Formular: Alle rechteckigen Felder werden korrekt zugeordnet, inklusive der Checkboxen. Vorname und Nachname bleiben auch hier unerkannt, da keine Kontur vorhanden ist. . 20
4.7 Telemetrie-Dashboard nach einer LLM-gestützten Verarbeitung mit Gemini 2.5 Flash: 9 von 9 Feldern zugeordnet, durchschnittliche Konfidenz 100 %, Verarbeitungszeit 17.911 ms, Kosten $0,0010, 4.321 interne Reasoning-Tokens 24

5.1 Template-Übersicht: Alle angelegten Templates mit Vorschaubild und direkten Aktionen. Links *New Template* für manuelle Erstellung, rechts *Generate Template* für automatisierte Generierung. 31
5.2 Formular zur automatisierten Template-Generierung: Name, Quelldatei und Erkennungsmethode. Bei Auswahl von *AI-Powered Detection* erscheint die Providerauswahl und eine Live-Verbindungsprüfung. 32
5.3 Template-Detailansicht nach automatisierter Erkennung: Statusmeldung über verwendete Methode und Erkennungsqualität, Metadaten sowie aufklappbare Sektionen für Vorschau, Prediction-Bild, Anhänge, Felder und KI-Telemetrie. 33
5.4 Eingabeformular zur PDF-Generierung aus dem Template *test_rechnung*: Die erkannten Feldnamen dienen als Beschriftungen. Unten: Attachment-Checkbox, Dataset-Auswahl und Dateiname. 35
5.5 Dataset-Erstellung: Tabellarische Eingabe von Feldwerten und Aliases. Im Beispiel sind *nachname* mit dem Wert *Salomon* und den Aliases *name;surname* sowie *e-mail* mit dem Alias *mail* befüllt. 36
5.6 Dataset-Übersicht: Drei gespeicherte Datensätze mit Quelltemplates und Feldanzahl. Über *Fill Template* mit Dropdown wird ein Dataset direkt auf ein beliebiges Template angewendet. 37
5.7 History-Ansicht: Chronologische Liste aller PDF-Generierungen mit Dateinamen, Datum und Direktaktionen für Details, Preload, Field Preview und Dataset-Erstellung. 37

Tabellenverzeichnis

4.1 Direktvergleich: Traditioneller Algorithmus vs. LLM (Gemini 2.5 Flash) auf beiden Testformularen 26
4.2 Kosten- und Zeitvergleich der drei Provider (Schätzwerte für einseitiges Formular mit ca. 10 Feldern) 27

1 Einleitung

1.1 Problemstellung und Motivation

In der heutigen teil-digitalisierten Welt stellt die effiziente und robuste Verarbeitung von Dokumenten eine zentrale Herausforderung für Unternehmen, Behörden und Privatpersonen da. Das manuelle erstellen von interaktiven PDF's und die wiederholte Dateneingabe kosten viel Zeit und sind fehleranfällig. Das Projekt bietet einen Ansatz dieses Problem zu lösen. Es bezieht sich auf PDF Dateien. Diese werden für die Verarbeitung in Bilder umgewandelt. Der Ansatz funktioniert auch mit eingescannten Dokumenten, wobei zu beachten ist, dass bei eingescannten Dokumenten der Text mithilfe von OCR erkannt werden muss, während er aus PDF's meistens extrahiert werden kann. Die Texterkennung mittels OCR ist relativ fortgeschritten, jedoch fehleranfälliger als das direkte extrahieren.
Die Herangehensweise an die PDF-Formularverarbeitung erfordert mehrere Schritte:
Zunächst muss die Struktur eines Formulars analysiert, die Eingabefelder identifiziert und deren Position bestimmt werden. Anschließend müssen die einzelnen interaktiven Felder positioniert und ausgefüllt werden. Bei der Verwendung müssen Daten häufig redundant in Formulare eingegeben werden, was zeitaufwändig und fehleranfällig ist.
Die Motivation für diese Arbeit besteht darin, diese Schritte durch den Einsatz von Computer Vision Technologien sowie Sprachmodelle zu erleichtern. Durch die automatische Erkennung von Formularfeldern und deren intelligenter Verwaltung in einem integrierten Management-System kann den Verarbeitungsprozess erheblich verbessern und die Fehleranfälligkeit reduzieren.

1.2 Forschungsfrage

Die zentrale Forschungsfrage dieser Arbeit lautet:
Wie können Computer Vision Algorithmen zur automatischen Erkennung und Klassifizierung von Eingabefeldern in PDF-Formularen entwickelt werden und welche technischen Herausforderungen bestehen?
Diese übergeordnete Fragestellung gliedert sich in mehrere Teilfragen auf:

1. **Algorithmische Grundlagen:** Welche Computer Vision Techniken eignen sich für die Erkennung von Eingabefeldern in PDF-Dokumenten? Wie können Bildverarbeitungsmethoden (Kantenerkennung, Konturdetektion) mit modernen Ansätzen (OCR, räumliche Analyse) kombiniert werden, um robuste Ergebnisse zu erzielen?

2. **Label-Feld-Zuordnung:** Wie kann ein Algorithmus entwickelt werden, der die semantische Beziehung zwischen Textlabels und Eingabefeldern korrekt identifiziert? Welche räumlichen und kontextuellen Merkmale sind für eine zuverlässige Zuordnung entscheidend?

Die Beantwortung dieser Forschungsfrage erfolgt durch die Entwicklung eines Prototyps, der als Proof-of-Concept die technische Realisierbarkeit der automatischen Formularfelderkennung demonstriert.

1.3 Aufbau der Arbeit

Die Arbeit besteht inkl. Einleitung aus sechs Kapiteln.

Kapitel 2 – Systemarchitektur gibt einen Überblick über das gesamte System. Es beschreibt die verwendeten Technologien, die Datenbankstruktur und wie die einzelnen Komponenten miteinander verbunden sind. Dieses Kapitel bildet die technische Grundlage für alle nachfolgenden Kapitel.

Kapitel 3 – Computer Vision Algorithmus erklärt den klassischen, regelbasierten Ansatz zur Formularfelderkennung. Es wird beschrieben, wie ein PDF-Dokument in ein Bild umgewandelt, die Felder geometrisch erkannt und anschließend den richtigen Labels zugeordnet werden.

Kapitel 4 – Semantische Label-Feld-Zuordnung durch KI-Modelle knüpft an die Grenzen der klassischen Herangehensweise an. Es zeigt, wo der klassische Algorithmus scheitert, und erklärt wie Large Language Models dieses Problem lösen können. Außerdem wird die Anbindung verschiedener LLM-Anbieter beschrieben und die Ergebnisse beider Ansätze verglichen.

Kapitel 5 – Web-Interface und Management-System beschreibt die Benutzeroberfläche, über die alle Funktionen des Systems zugänglich sind. Der Nutzer kann hier Templates erstellen, PDFs generieren und Datasets verwalten.

Kapitel 6 – Fazit und Ausblick bewertet die Ergebnisse, beantwortet die Forschungsfrage und zeigt mögliche Verbesserungen auf.

2 Systemarchitektur

Bevor die einzelnen Algorithmen und Funktionen des Systems im Detail beschrieben werden, gibt dieses Kapitel einen Überblick über die gesamte Architektur. Es beantwortet die Frage, aus welchen Teilen das System besteht, wie sie zusammenhängen und welche technischen Entscheidungen dabei getroffen wurden.
Abschnitt 2.1 beschreibt den Technologie-Stack und gibt einen ersten Überblick über die Komponenten. Abschnitt 2.2 erläutert das Datenbankdesign inklusive Entity-Relationship-Modell, Datenfluss und Schnittstellen.

2.1 Überblick der Komponenten & Technologie-Stack

Das entwickelte System basiert auf einer mehrschichtigen Architektur und folgt dem Model-View-Template (MVT) Pattern des Django-Frameworks, erweitert dieses aber noch um Module für Computer Vision und PDF-Verarbeitung. Die Wahl des Web-Frameworks Django ist in dieser Arbeit nebensächlich und es hätte auch ein anderes Web-Framework den Zweck erfüllt. Django übernimmt hier die Rolle der Datenverwaltung sowie das darstellen der Nutzeroberflächen im Web. Die Grundlage für Django sowie die anderen genutzten Computer Vision und LLM Technologien bietet die Programmiersprache Python. Der Vorteil liegt vor allem in der hohen Verfügbarkeit von bestimmten Bibliotheken, besonders bezogen auf Computer Vision. Es wäre aber auch möglich gewesen das System in einer anderen Sprache zu entwickeln.
Die folgenden Python Bibliotheken wurden für den Computer Vision Algorithmus genutzt:

Python Bibliotheken	
django	Web-Framework
cv2	OpenCV Computer Vision
pytesseract	OCR Engine
numpy	zum berechnen der Dimensionen
matplotlib.pyplot	zum darstellen der Felder
pdf2image	zum umwandeln von PDFs zu Bildern
pdfminer	zum extrahieren von Text aus PDFs
reportlab	zum erstellen von PDFs
PyPDF2	zum laden und erstellen von PDFs

2.2 Datenbankdesign

Das Datenbankschema besteht aus vier Hauptentitäten, die über Foreign-Key-Beziehungen verknüpft sind. Die Verwendung von UUIDs sorgt dafür dass Kollisionen theoretisch extrem unwahrscheinlich sind und praktisch nicht vorkommen.

2.2.1 Entity-Relationship-Modell

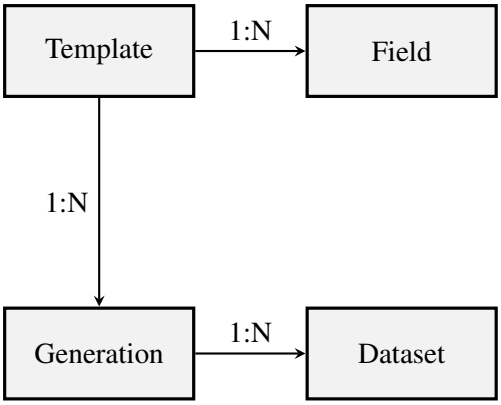

Abbildung 2.1: Entity-Relationship-Diagramm der Datenbank
a

2.2.2 Datenfluss-Architektur

Der Verarbeitungsfluss folgt einem Pipeline-Muster:

1. **Upload-Phase:** HTTP-Request mit PDF-Datei erreicht `views.generate_template`
2. **Persistierung:** Django's `default_storage` speichert die Datei unter `media/source_files/` mit UUID-Präfix
3. **CV-Verarbeitung:** Aufruf von `CVprod.process_to_template()` mit Dateipfad
4. **Koordinatentransformation:** Konvertierung von OpenCV-Koordinaten zu Reportlab-Koordinaten
5. **Template-Erstellung:** `Pdf_template` instanziiert mit Felddaten-Dictionary
6. **Datenbank:** ORM-basierte Speicherung von Template und Fields
7. **Response:** Redirect zur Template-Detailansicht

2.2.3 Schnittstellendesign

Die Schnittstelle zwischen Web-Interface und CV-Modul sieht folgendermaßen aus:
Input-Schnittstelle:

```
CVprod.process_to_template(
pdf_path: str,
use_llm: bool = False,
llm_provider: str = 'openai'
) -> Tuple[List[Dict], PIL.Image, np.array]
```

Output-Datenstruktur:

```
fields = [{
        "name": str,            # Feldbezeichnung
        "field_type": str,      # Feldtyp
        "page_index": int,      # Seitenindex
        "pos_x": float,         # X-Position
        "pos_y": float,         # Y-Position
        "font_size": int,       # Schriftgroesse
        "font": str             # Schriftart
}, ...]
```

2.2.4 Konfigurationsmanagement

Das System nutzt Umgebungsvariablen für Konfigurationen:

- **Pfadkonfiguration:** `TEMPLATE_PATH`, `OUTPUT_PATH`
- **LLM-Settings:** `USE_LLM_FOR_MATCHING`, `LLM_PROVIDER`
- **API-Schlüssel:** Sichere Speicherung in `.env`
- **Tesseract-Pfad:** Systemabhängige OCR-Engine-Lokation

Die `python-dotenv` Bibliothek lädt diese Konfigurationen zur Laufzeit, was eine Trennung von Code und Konfiguration ermöglicht und die Portabilität zwischen Entwicklungs- und Produktionsumgebungen gewährleistet. Außerdem wird sichergestellt dass sensible Daten wie API-Keys nicht über ein Git Repository veröffentlicht werden, indem die .env und andere sensible Dateien in der .gitignore angegeben wird.

3 Computer Vision Algorithmus

Auf Basis der in Kapitel 2 beschriebenen Systemarchitektur wird in diesem Kapitel der erste der beiden Erkennungsansätze vorgestellt: ein klassischer Computer-Vision-Algorithmus, der ohne externe Dienste auskommt und rein auf geometrischer Bildanalyse basiert.
Der Algorithmus verarbeitet ein PDF-Dokument in mehreren Stufen. Abschnitt 3.2 beschreibt, wie die PDF zunächst in ein Rasterbild umgewandelt und der enthaltene Text extrahiert wird. Abschnitt 3.3 erläutert die Bildvorverarbeitung und die eigentliche Felderkennung durch Kantendetektion und Konturanalyse. Abschnitt 3.4 beschreibt, wie die erkannten Felder anschließend den richtigen Labels zugeordnet werden. Abschnitt 3.5 zeigt schließlich, wie die Ergebnisse ausgegeben und visualisiert werden.

3.1 Überblick der Verarbeitungspipeline

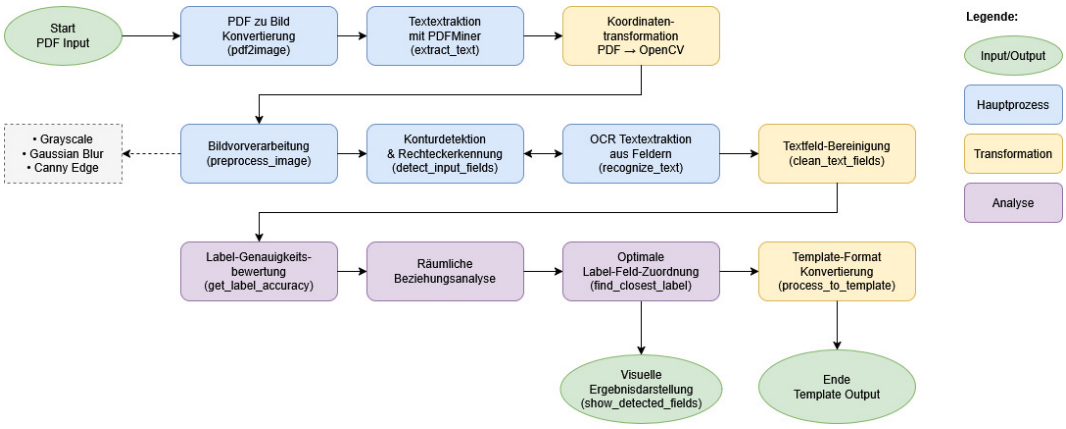

Abbildung 3.1: Vollständiger Datenfluss des Computer Vision Algorithmus mit farblicher Kodierung der Verarbeitungsebenen

Der entwickelte Computer Vision Algorithmus implementiert eine mehrstufige Verarbeitungspipeline zur automatischen Erkennung und Klassifizierung von Formularfeldern in PDF-Dokumenten. Die Pipeline gliedert sich in vier Hauptphasen: (1) PDF-Verarbeitung und Textextraktion, (2) Bildvorverarbeitung und Felderkennung, (3) Intelligente Label-Zuordnung und (4) Ausgabe-Generierung.
Abbildung 3.1 visualisiert den vollständigen Datenfluss des Algorithmus. Die farbliche Kodierung unterscheidet dabei zwischen verschiedenen Verarbeitungsebenen: Eingabe/Ausgabe-Operationen (grün), Verarbeitungsschritte (blau), Transformations- und Konvertierungsprozesse (gelb) und Analyseschritte (violett).
Die Pipeline beginnt mit der Verarbeitung des PDF-Dokuments auf zwei Wegen: der pixelbasierten Bildanalyse mittels OpenCV und der strukturierten Textextraktion durch PDF-Miner. Dadurch können visuelle und semantische Informationen kombiniert werden. Ein wichtiger Aspekt ist die Koordinatentransformation zwischen den unterschiedlichen Koordinatensystemen, die in Abschnitt 3.2.3 behandelt wird.

3.2 PDF-Verarbeitung und Textextraktion

PDF-Dokumente sind primär für die visuelle Darstellung konzipiert und nicht für die maschinelle Verarbeitung. Das Portable Document Format speichert Inhalte als Kombination aus Vektorgrafiken, eingebetteten Schriften und Rastergrafiken. Um sowohl die visuelle Struktur als auch den Text zu analysieren, verfolgt die Pipeline einen dualen Ansatz: Die Konvertierung in Rastergrafiken ermöglicht Bildverarbeitungsoperationen, während die direkte Textextraktion Positionsinformationen der Textblöcke liefert.

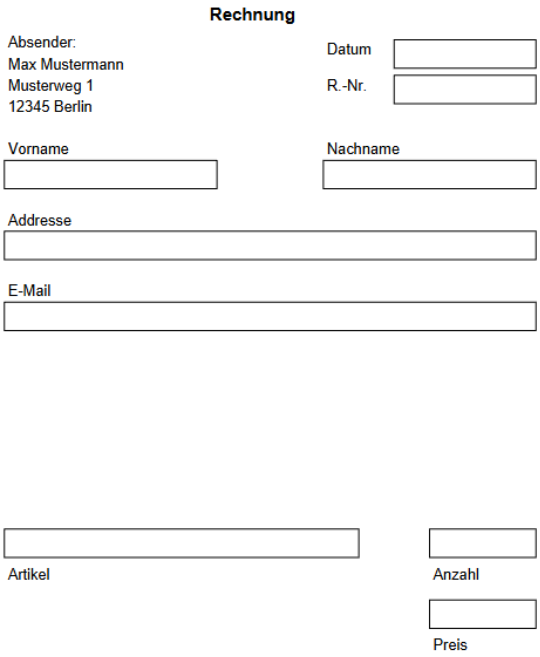

Abbildung 3.2: Beispiel PDF Formular (Rechnung, wurde aus Platzgründen unten etwas gekürzt)

3.2.1 PDF zu Bild Konvertierung

Die Konvertierung von PDF-Seiten in Rasterbilder bildet die Grundlage für die nachfolgenden Bildverarbeitungsschritte. Dafür wird die `pdf2image`-Bibliothek verwendet, die intern Poppler als Rendering-Engine nutzt.
Die Qualität der Rasterisierung hängt von der gewählten Auflösung in DPI (Dots Per Inch) ab. Die Standardauflösung von 200 DPI erzeugt für ein A4-Dokument ein Bild von circa 1654×2339 Pixeln. Diese Auflösung ist ein Kompromiss zwischen Erkennungsgenauigkeit und Verarbeitungszeit.
Der Zusammenhang zwischen PDF-Punkten und Pixeln ergibt sich aus der DPI-Definition. Ein PostScript-Punkt entspricht $\frac{1}{72}$ Zoll. Bei einer Rasterisierung mit d DPI gilt für die Pixelanzahl p pro PDF-Punkt:

$$p = \frac{d}{72} \approx 2.778 \text{ bei } d = 200 \text{ DPI} \tag{3.1}$$

Diese Beziehung erklärt den im Code verwendeten Skalierungsfaktor von 2.775, der geringfügig vom theoretischen Wert abweicht, was auf Rundungseffekte der Rendering-Engine zurückzuführen ist.

3.2.2 Strukturierte Textextraktion mit PDFMiner

Parallel zur Bildkonvertierung extrahiert PDFMiner die Textinhalte direkt aus der PDF-Struktur. Im Gegensatz zu OCR-basierter Texterkennung liefert dieser Ansatz exakte Positionsinformationen und den korrekten Textinhalt ohne Erkennungsfehler, da die Zeicheninformationen bereits im PDF kodiert sind.
Die PDF-Spezifikation definiert Text als Sequenz von Glyphen, die durch Zeichenoperatoren positioniert werden. PDFMiner aggregiert diese zu höheren Struktureinheiten:

- **LTChar**: Einzelnes Zeichen mit Bounding Box
- **LTTextLine**: Zeile zusammengehöriger Zeichen
- **LTTextBox**: Rechteckiger Bereich mit einer oder mehreren Zeilen
- **LTPage**: Gesamte Seite als Container für alle Layoutobjekte

Die `LAParams`-Klasse steuert die Gruppierung von Textfragmenten.
Der `line_margin`-Parameter definiert den maximalen vertikalen Abstand, bei dem Zeichen noch derselben Zeile zugeordnet werden. Der `word_margin` bestimmt analog die horizontale Gruppierung. Diese Parameter beeinflussen direkt, wie Textblöcke segmentiert werden.
Der folgende JSON Output zeigt in welcher Form PDFMiner gefunden Objekte zurückgibt. Aus Platzgründen wurden nur die ersten 4 erkannten Objekte dargestellt.

Listing 3.1: Extrahierte Daten aus der OCR-Pipeline

```
[
{
        "id": "afc6fa441929",
        "text": "Rechnung",
        "bbox": [691, 101, 881, 140],
        "accuracy": 0,
        "page_index": 0,
        "state": null
},
{
        "id": "701e7457f001",
        "text": "Datum",
        "bbox": [951, 183, 1049, 216],
        "accuracy": 0,
        "page_index": 0,
        "state": null
},
{
        "id": "bd60d5a1a1c5",
        "text": "R.-Nr.",
        "bbox": [951, 261, 1040, 294],
        "accuracy": 0,
        "page_index": 0,
        "state": null
},
{
        "id": "03716914800f",
        "text": "Nachname",
        "bbox": [951, 403, 1112, 436],
```

```
                "accuracy": 10,
                "page_index": 0,
                "state": null
        }
        ]
```

3.2.3 Koordinatensystem-Transformation

Die Transformation zwischen PDF-Koordinaten und OpenCV-Pixelkoordinaten ist notwendig, da beide Systeme unterschiedliche Konventionen verwenden. PDF hat den Koordinatenursprung in der linken unteren Ecke mit Einheiten in Punkten ($\frac{1}{72}$ Zoll). OpenCV hingegen platziert den Ursprung in der linken oberen Ecke mit Pixeln als Einheit. Die Transformation erfordert daher eine Skalierung und eine Spiegelung der y-Achse.
Sei (x_{pdf}, y_{pdf}) ein Punkt in PDF-Koordinaten, h_{image} die Bildhöhe in Pixeln und s der Skalierungsfaktor. Die Transformation in OpenCV-Koordinaten (x_{cv}, y_{cv}) lässt sich folgendermaßen darstellen:

$$\begin{bmatrix} x_{cv} \\ y_{cv} \end{bmatrix} = \begin{bmatrix} s & 0 \\ 0 & -s \end{bmatrix} \begin{bmatrix} x_{pdf} \\ y_{pdf} \end{bmatrix} + \begin{bmatrix} 0 \\ h_{image} \end{bmatrix} \tag{3.2}$$

3.3 Bildvorverarbeitung und Felderkennung

Die Felderkennung nutzt klassische Computer Vision Methoden. Im Gegensatz zu Deep-Learning-Ansätzen wie LayoutLM[1] erfordern diese Methoden keine Trainingsdaten und sind deterministisch. Die Verarbeitungskette besteht aus: Rauschunterdrückung, Kantendetektion, Konturfindung und geometrischer Analyse.

3.3.1 Preprocessing-Pipeline

Die Bildvorverarbeitung transformiert das RGB-Eingabebild in eine für die Kantendetektion optimierte Darstellung. Wie in Abbildung 3.6 dargestellt, durchläuft das Bild drei Phasen: Graustufenkonvertierung, Gaußsche Glättung und Canny-Kantendetektion.

Graustufenkonvertierung und Glättung

Die Reduktion auf einen Graustufenkanal eliminiert farbbasierte Variationen. Die Konvertierung folgt dem ITU-R BT.601 Standard:[2]

$$Y = 0.299 \cdot R + 0.587 \cdot G + 0.114 \cdot B \tag{3.3}$$

Anschließend wird eine Gaußsche Glättung mit einem 5×5 Kernel angewandt. Dies reduziert hochfrequentes Rauschen (wie Scanner-Artefakte oder Papierstruktur), das die Kantendetektion andernfalls mit falschen Positivmeldungen stören würde.

Canny-Kantendetektion

Der Canny-Algorithmus[3] ist ein mehrstufiger Kantendetektor. Im ersten Schritt berechnet der Sobel-Operator[4] die Gradientenmagnitude M und -richtung θ:

[1] Vgl. Xu et al., 2020
[2] Vgl. International Telecommunication Union, 2011
[3] Vgl. Canny, 1986, S. 679.
[4] Vgl. Sobel/Feldman, 1968

$$M = \sqrt{G_x^2 + G_y^2}, \quad \theta = \arctan\left(\frac{G_y}{G_x}\right) \tag{3.4}$$

Anschließend verdünnt die sogenannte *Non-Maximum Suppression* die erkannten Strukturen auf exakt ein Pixel Breite (sichtbar in Abb. 3.6, rechts). Die abschließende Kantenverfolgung durch Hysterese nutzt zwei empirisch ermittelte Schwellwerte ($T_{low} = 75$, $T_{high} = 200$), um Rauschen endgültig zu verwerfen und nur geschlossene Objektgrenzen zu erhalten.

3.3.2 Konturdetektion und Rechteckerkennung

Die isolierten Kantenpixel werden im nächsten Schritt durch den Suzuki-Abe-Algorithmus[5] zu Vektorpolygonen (Konturen) gruppiert. Da Formularelemente in der Regel rechteckig sind, prüft der Algorithmus jede Kontur auf ihre Geometrie.

Douglas-Peucker Polygon-Approximation

Reale Kanten in Scans sind selten perfekte Linien. Der Douglas-Peucker Algorithmus[6] vereinfacht Polygonzüge durch iterative Punktreduktion.

Listing 3.2: Kernlogik der Kontur-Approximation und Filterung

```
# Douglas-Peucker Polygon-Approximation pro Kontur
epsilon = 0.02 * cv2.arcLength(cnt, True)
approx = cv2.approxPolyDP(cnt, epsilon, True)

# Validierung: Nur Vierecke (4 Eckpunkte) zulassen
if len(approx) == 4:
x, y, w, h = cv2.boundingRect(approx)

# Rauschen filtern: Mindestgroesse vorausgesetzt
if w >= min_width and h >= min_height:
input_fields.append({'bbox': [x, y, x+w, y+h]})
```

Der Toleranzparameter $\epsilon = 0.02 \cdot$ Umfang skaliert mit der Konturgröße. Der Algorithmus reduziert Rauschen auf exakt vier mathematische Eckpunkte.

Rechteckvalidierung und Größenfilterung

Wie in Listing 3.2 ersichtlich, werden nur Polygone mit genau vier Ecken als Formularfelder in Betracht gezogen. Abbildung 3.9 visualisiert jedoch, dass diese geometrische Eigenschaft allein nicht ausreicht.
Ohne weitere Filterung (Abb. 3.9, links) erkennt der Algorithmus auch Buchstabenfragmente, Logos oder Scanner-Artefakte als Vierecke. Erst die Heuristik, dass ein valides Eingabefeld mindestens 40×20 Pixel groß sein muss, extrahiert robust die eigentlichen Zielstrukturen (rechts). Abschließend prüft Tesseract OCR[7], ob die verbliebenen Felder bereits Textinhalte aufweisen.

[5] Vgl. Suzuki/Abe, 1985, S. 32.
[6] Vgl. Douglas/Peucker, 1973
[7] Vgl. Smith, 2007

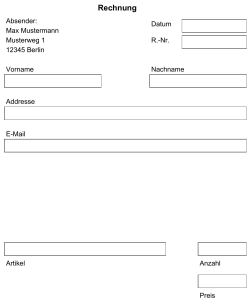

Abbildung 3.3: Graustufenkonvertierung

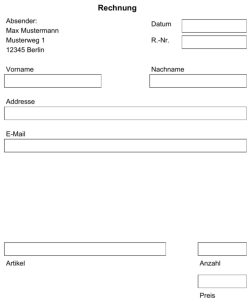

Abbildung 3.4: Gaußsche Glättung

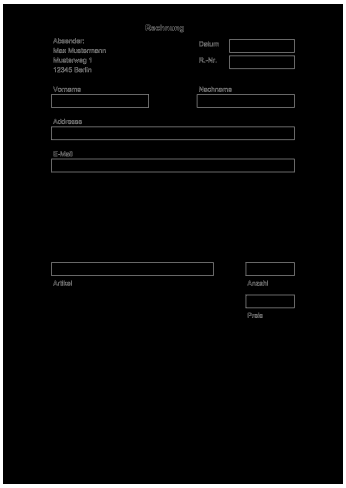

Abbildung 3.5: Canny-Kantendetektion

Abbildung 3.6: Visualisierung der Preprocessing-Schritte zur Extraktion definierter Formularfeld-Kanten.

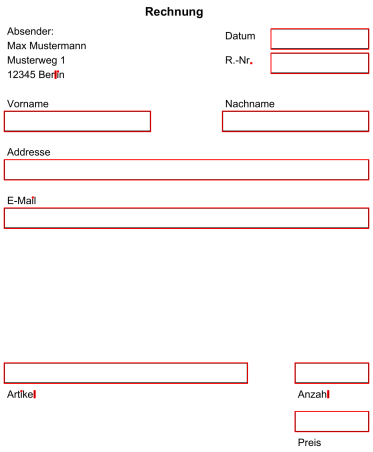

Abbildung 3.7: Alle erkannten 4-Punkt-Polygone

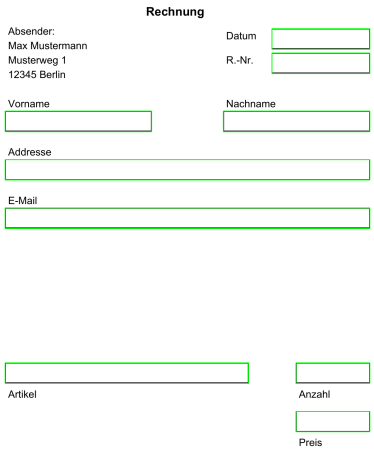

Abbildung 3.8: Nach Filterung ($w \geq 40$, $h \geq 20$)

Abbildung 3.9: Die Größeneinschränkung eliminiert erfolgreich Störartefakte und Buchstaben, die zufällig eine viereckige Struktur aufweisen.

3.4 Intelligente Label-Zuordnung

Nach der Erkennung der leeren Formularfelder (durch OpenCV) und der Textblöcke (durch PDFMiner) muss die semantische Zuordnung erfolgen: Welcher Text beschreibt welches Eingabefeld?

3.4.1 Räumliche Beziehungsanalyse

Die Bestimmung der Beziehung zwischen einem Textblock und einem Feld erfolgt über die Analyse der jeweiligen Bounding Boxes. Die Bedingung für „Label L ist oberhalb von Feld F“ lässt sich formal definieren. Sei $F = [f_{x0}, f_{y0}, f_{x1}, f_{y1}]$ und $L = [l_{x0}, l_{y0}, l_{x1}, l_{y1}]$:

$$l_{y1} \leq f_{y0} \quad \wedge \quad f_{x0} \leq \frac{l_{x0} + l_{x1}}{2} \leq f_{x1} \tag{3.5}$$

Die erste Bedingung fordert, dass das Label vertikal über dem Feld liegt. Die zweite stellt sicher, dass die horizontale Mitte des Labels im X-Bereich des Feldes zentriert ist, um Fehler bei benachbarten Spalten zu vermeiden. Zudem wird geprüft, ob ein Text als Platzhalter vollständig in einem Feld liegt:

$$L \subseteq F \iff [l_{x0}, l_{x1}] \subseteq [f_{x0}, f_{x1}] \wedge [l_{y0}, l_{y1}] \subseteq [f_{y0}, f_{y1}] \tag{3.6}$$

3.4.2 Label-Genauigkeitsbewertung

Nicht jeder Textblock (z.B. Überschriften oder Fußnoten) ist ein valides Label. Das System nutzt eine Scoring-Heuristik zur Validierung von Label-Kandidaten:

Listing 3.3: Heuristische Validierung von Label-Kandidaten

```
# Doppelpunkt-Heuristik: Labels enden oft mit ':'
if label_text.endswith(':'):
accuracy += 10

# Keyword-Matching gegen definierte Dictionaries
if label_text in common_labels:
accuracy += 10
```

3.4.3 Greedy Label-Feld-Zuordnung

Die finale Zuordnung ist ein bipartites Optimierungsproblem. Die Implementierung nutzt einen performanten Greedy-Ansatz: Sie priorisiert die „Within“-Relation (Gleichung 3.6) und ordnet verbleibende Felder dem geometrisch am nächsten gelegenen Label zu. Abbildung 3.10 visualisiert diese berechneten Vektoren.

3.5 Ausgabe-Generierung und Visualisierung

Nachdem die Label-Feld-Paare feststehen, müssen die Ergebnisse in zwei unterschiedliche Ausgabeformate überführt werden. Zum einen werden die erkannten Felder in das interne Template-Format konvertiert, das die nachgelagerte `Pdf_template`-Klasse für die spätere PDF-Generierung erwartet. Dabei werden die OpenCV-Pixelkoordinaten in ReportLab-Punkte umgerechnet, da ReportLab ein eigenes Koordinatensystem mit dem Ursprung in der linken unteren Ecke verwendet. Zum anderen wird ein annotiertes Vorschaubild erzeugt, auf dem

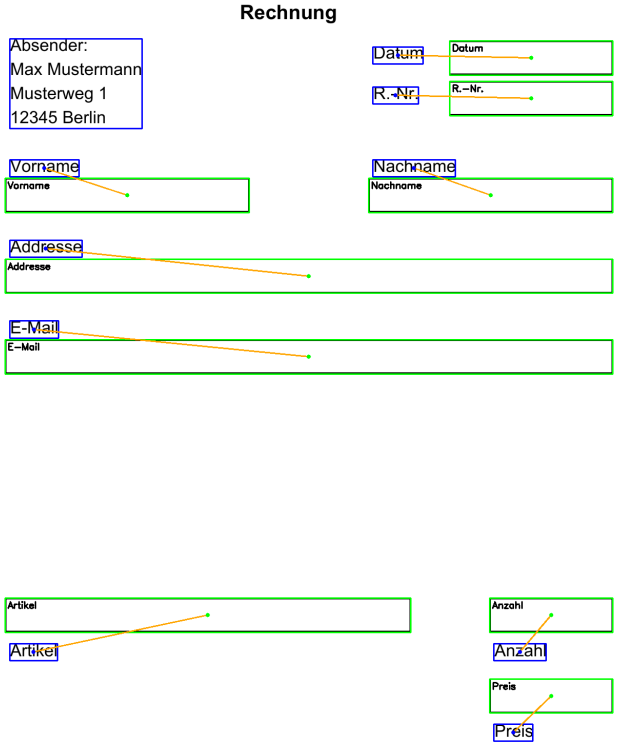

Abbildung 3.10: Jedes Eingabefeld (grün) wird seinem semantisch nächsten Label-Text (blau) zugeordnet.

erkannte Felder (grün) und Textblöcke (blau) eingezeichnet sind. Dieses Bild dient der visuellen Qualitätskontrolle und der Nutzer kann im Web-Interface auf einen Blick erkennen, ob alle Felder korrekt erkannt und benannt wurden.
Abschnitt 3.5.1 beschreibt die Koordinatenumrechnung und das Ausgabeformat.
Abschnitt 3.5.2 erläutert die Erstellung des Annotationsbilds.

3.5.1 Template-Format-Konvertierung

Die OpenCV-Pixelkoordinaten müssen in Reportlab-Punkte zurückgerechnet werden. Da Scans oft minimale optische Krümmungen aufweisen, greift hier eine quadratische Korrekturformel:

Listing 3.4: Quadratische Verzerrungskorrektur für das Zielsystem

```
# Quadratische Korrektur: staerkere Anpassung
# je weiter vom Zentrum
pos_y = pos_y + ((pos_y - original_height / 2) * b)
pos_y = (original_height - pos_y) * scale

pos_x = pos_x + ((pos_x - original_width / 2) * b)
pos_x = pos_x * scale + x_offset
```

Die zugrundeliegende mathematische Korrektur lautet:

$$x' = x + b \cdot \left(x - \frac{w}{2}\right) \tag{3.7}$$

Wobei $b = 0.03$ einen empirisch ermittelten Parameter darstellt. Die Korrektur wirkt stark an den Bildrändern und ist im Bildzentrum null.

3.5.2 Visuelle Ergebnisdarstellung

Zur Validierung und Nachvollziehbarkeit generiert der Algorithmus ein abschließendes annotiertes Output-Bild (siehe Abbildung 3.11), welches das Originaldokument mit den semantisch gelabelten Feldern überlagert.

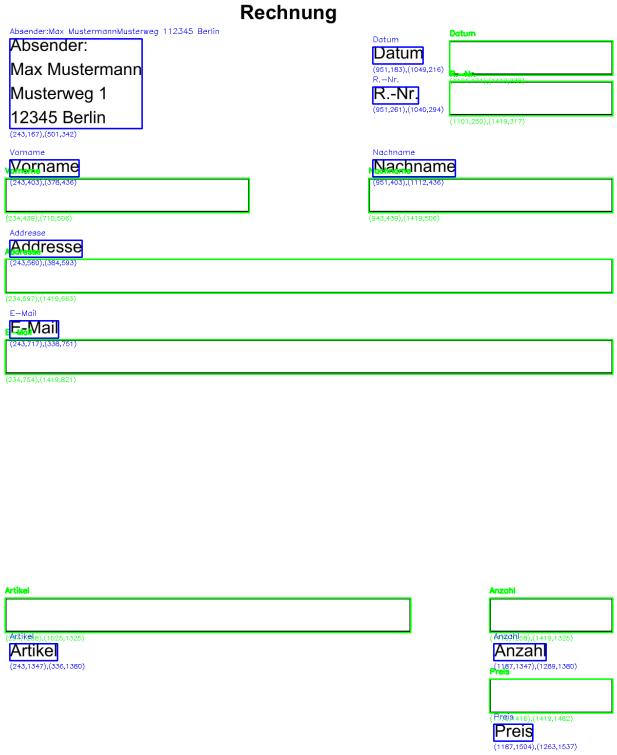

Abbildung 3.11: Finales Output: Erfolgreich extrahierte und benannte Formularfelder (grün), bereit zur Verarbeitung durch die nachfolgende Template-Engine.

4 Semantische Label-Feld-Zuordnung durch KI-Modelle

Die rein geometrische Analyse, wie sie in Kapitel 3 beschrieben wird, liefert für standardisierte Formulare gute Ergebnisse, stößt aber bei komplexen Dokumentstrukturen an ihre Grenzen. Um diese Grenzen zu überwinden, wurde eine zweite Zuordnungskomponente entwickelt, die auf Large Language Models (LLMs) basiert. Dieses Kapitel dokumentiert die Motivation, die konkrete Umsetzung und die Unterschiede zwischen den beiden Ansätzen.
Abschnitt 4.1 zeigt anhand von zwei konkreten Beispielen, wo und warum der geometrische Ansatz versagt. Abschnitt 4.2 erklärt die theoretischen Grundlagen von LLMs, soweit sie für die Aufgabe relevant sind. Abschnitt 4.3 beschreibt die Systemarchitektur und Designentscheidungen. Abschnitt 4.4 behandelt das Prompt Engineering. Abschnitt 4.5 vergleicht den LLM gestützten Ansatz mit dem traditionellen.

4.1 Motivation: Wo der geometrische Ansatz scheitert

Der in Kapitel 3 entwickelte Algorithmus basiert auf räumlichen Heuristiken: Labels befinden sich typischerweise links oder oberhalb ihres Feldes, der Abstand zwischen Label und Feld sollte möglichst gering sein, und Überschneidungen zwischen Label-Bounding-Box und Feld-Bounding-Box deuten auf direkte Zugehörigkeit hin. Diese Regeln sind bewusst einfach gehalten, weil einfache Heuristiken bei standardisierten Formularen ausreichend zuverlässig arbeiten und dabei kaum Rechenzeit benötigen. Bei komplexeren Dokumentstrukturen führen dieselben Heuristiken jedoch zu fehlerhaften Zuordnungen. Die folgenden zwei Beispiele zeigen unterschiedliche Klassen von Layoutproblemen, an denen der geometrische Ansatz scheitert.

4.1.1 Beispiel 1: Auftragsformular

Abbildung 4.1 zeigt ein Auftragsformular mit drei Abschnitten, die verschiedene Layoutprobleme kombinieren. Der Abschnitt *Kontaktdaten* enthält ein zweispaltiges Layout mit *Geb.-Datum:* links und *Tel.-Nr. (mobil):* rechts, die jeweils ihr eigenes Feld direkt darunter haben. Der *Bestellmatrix*-Abschnitt enthält drei Spalten mit je einem Label oben (*Artikelnummer*, *Menge*, *Einzelpreis*) und zwei Zeilen von Feldern darunter – die Felder der zweiten Zeile teilen sich die Labels der ersten Zeile, obwohl kein weiteres Label vorhanden ist. Der Problemfall liegt im Abschnitt *Auftragsdetails*: Das Label *Kundennummer:* steht links, daneben ist ein Feld, und rechts davon erscheint das Label ← *R.-Datum*. Das zugehörige Feld für das Rechnungsdatum liegt also links vom Label.
Abbildung 4.2 zeigt das konkrete Scheitern des traditionellen Algorithmus. Da der Algorithmus die Felder von unten nach oben zuordnet, entstehen durch die mehrzeiligen Felder Folgefehler. Der Algorithmus ordnet fälschlicherweise die unterste Zeile der mehrzeiligen Felder zu den am besten passenden Labeln zu: *Auftragsdetails, Menge* und *Einzelpreis*. Die dazu eigentlich gehörenden Felder werden dann als nächstes den ihren am besten passenden Labeln zugeordnet (von links nach rechts): *3. Bestellmatrix* (welches fälschlicherweise

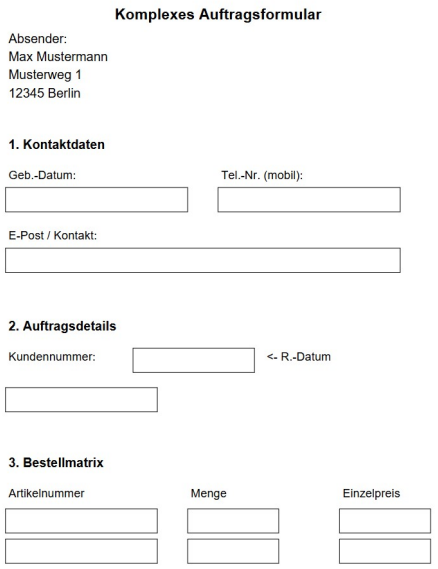

Abbildung 4.1: Komplexes Auftragsformular mit mehrspaltigen Abschnitten, geteilten Spaltenlabels und einem unkonventionell positionierten Label (← *R.-Datum*) rechts vom Feld

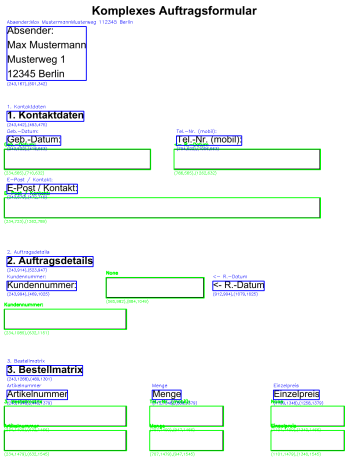

Abbildung 4.2: Traditioneller Algorithmus auf dem Auftragsformular: Das Feld für R.-Datum erhält `None`, da sein Label rechts davon steht. Das darunter liegende Feld wird fälschlicherweise mit *Kundennummer:* beschriftet.

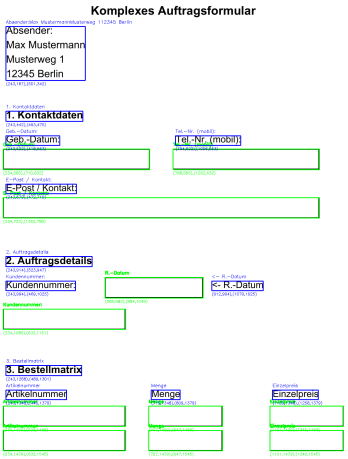

Abbildung 4.3: LLM-Ansatz (Gemini 2.5 Flash) auf demselben Formular: Alle 9 Felder werden korrekt zugeordnet, inklusive *R.-Datum* und der Tabellenfelder der zweiten Zeile.

nicht als Abschnittstitel erkannt wird), *Tel.-Nr. (mobil)* und *None* (Es gibt kein passendes Label mehr). Das Feld *Kundennummer* wird als erstes richtig zugeordnet, ←*R.-Datum* wird aber aufgrund der Heuristik, dass Label nicht rechts von ihren Feldern stehen dürfen, fälschlicherweise einem Feld im 1. Abschnitt zugeordnet.
Abbildung 4.3 zeigt den direkten Vergleich: Das Modell ordnet alle 9 Felder korrekt zu. Das Modell interpretiert den Pfeil im Labelnamen ← *R.-Datum* als expliziten Positionshinweis und erkennt aus dem semantischen Kontext des Formulars, dass es sich um ein Datumsfeld handelt. Die Tabellenfelder der zweiten Zeile werden korrekt mit den Labels der ersten Zeile verknüpft, da das Modell das Spaltenprinzip der Bestellmatrix als Ganzes versteht.

4.1.2 Beispiel 2: Bewerbungsformular

Das zweite Beispiel zeigt einen anderen Typ von Layoutproblemen, der im Auftragsformular nicht auftritt: Felder ohne rechteckige Umrandung sowie Checkboxen mit eng beieinanderliegenden Labels.

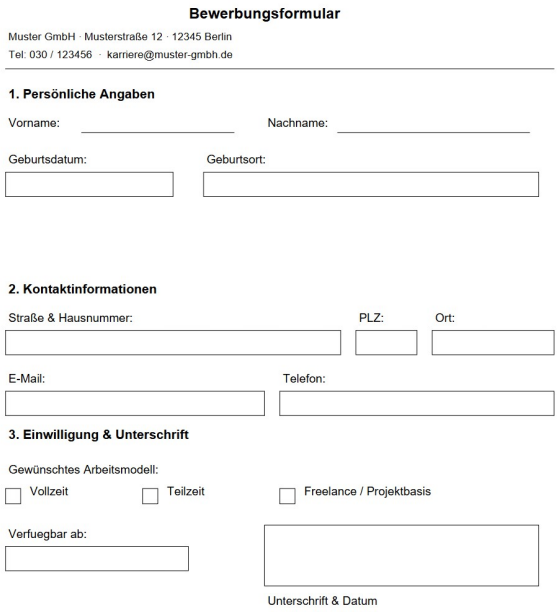

Abbildung 4.4: Bewerbungsformular mit drei verschiedenen Layoutfallen: Vorname und Nachname als Linienfelder (Abschnitt 1), drei eng beieinanderliegende Checkboxen mit Labels (Abschnitt 3) und ein Unterschriftsfeld mit Label unterhalb statt oberhalb.

Abbildung 4.4 zeigt ein Bewerbungsformular, das drei strukturell unterschiedliche Probleme enthält. In Abschnitt 1 sind die Felder für *Vorname* und *Nachname* nicht als Rechtecke, sondern als einfache Unterstriche (Linien) dargestellt – eine in klassischen Formularen verbreitete Konvention. In Abschnitt 3 stehen drei Checkboxen für das gewünschte Arbeitsmodell (*Vollzeit*, *Teilzeit*, *Freelance / Projektbasis*) eng nebeneinander, jeweils mit ihrem Label direkt rechts daneben. Ebenfalls in Abschnitt 3 befindet sich ein großes Unterschriftsfeld, dessen Label *Unterschrift & Datum* unterhalb des Feldes steht – entgegen der üblichen Konvention.
Abbildung 4.5 zeigt die Fehler des traditionellen Algorithmus. Da *Vorname* und *Nachname* als Linien dargestellt sind, findet OpenCV keine geschlossene Kontur, erkennt diese Felder

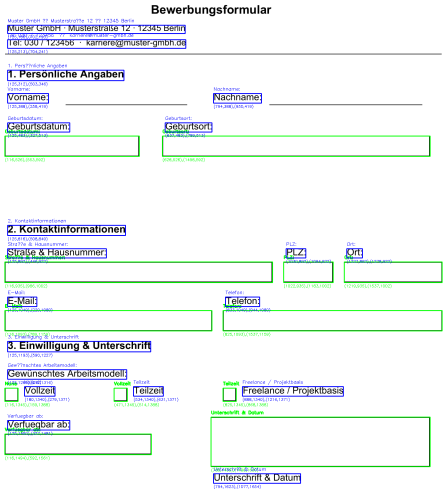

Abbildung 4.5: Traditioneller Algorithmus auf dem Bewerbungsformular: Vorname und Nachname werden nicht erkannt (keine Kontur), die Checkbox-Labels sind um eine Position verschoben zugeordnet.

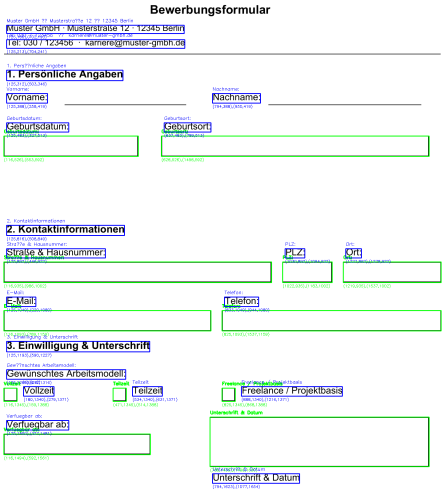

Abbildung 4.6: LLM-Ansatz (Gemini 2.5 Flash) auf demselben Formular: Alle rechteckigen Felder werden korrekt zugeordnet, inklusive der Checkboxen. Vorname und Nachname bleiben auch hier unerkannt, da keine Kontur vorhanden ist.

gar nicht und sie tauchen im Ergebnis nicht auf. Bei den Checkboxen ordnet der Algorithmus jedes Label der geometrisch nächsten Box zu. Da die Labels jedoch rechts neben ihrer jeweiligen Box stehen und die Boxen eng zusammenliegen, verschiebt sich die Zuordnung systematisch: *Vollzeit* erhält `None` (kein Label links oder oben), *Teilzeit* wird der Vollzeit-Box zugeordnet, *Freelance / Projektbasis* der Teilzeit-Box. Das Unterschriftsfeld erhält ebenfalls `None`, da der Algorithmus ausschließlich Labels oberhalb und links eines Feldes berücksichtigt – das unterhalb stehende Label wird nicht gefunden.
Abbildung 4.6 zeigt das LLM-Ergebnis: Gemini 2.5 Flash ordnet alle rechteckigen Felder korrekt zu. Die Checkbox-Labels werden richtig zugewiesen, da das Modell aus dem Dokumentkontext und der räumlichen Nähe erkennt, welches Label zu welcher Box gehört. Das Unterschriftsfeld wird korrekt als *Unterschrift & Datum* klassifiziert, weil das Modell dieses Muster aus dem Training kennt und die Position des Labels relativ zum Feld nicht als Ausschlusskriterium wertet. Die Linienfelder für *Vorname* und *Nachname* bleiben auch beim LLM-Ansatz unerkannt. Nicht weil die Zuordnung falsch wäre, sondern weil OpenCV diese Felder bereits in der vorgelagerten Erkennungsphase nicht als Felder identifiziert. Das LLM bekommt sie nicht als Kandidaten für Felder übergeben.

4.1.3 Ableitung der Fehlermuster

Aus den beiden Beispielen lassen sich drei Layoutmuster ableiten, bei denen der geometrische Ansatz fehlerhafte oder unvollständige Ergebnisse liefert:

1. **Mehrzeilige Felder:** Der Algorithmus kann ein Label immer nur einem Feld zuordnen. Bei Formularen mit mehreren Feldern unter einem gemeinsamen Spalten-Label entstehen Folgefehler, bei denen nachfolgende Felder falsch oder gar nicht zugeordnet werden.

2. **Label rechts vom Feld:** Der Algorithmus sucht Labels ausschließlich links und oberhalb eines Feldes. Unkonventionelle Layouts werden ignoriert und das Feld erhält `None` bzw. wird Falsch zugeordnet.

3. **Felder ohne rechteckige Umrandung:** Linienfelder (Unterstriche statt Boxen) erzeugen keine geschlossene Kontur, die OpenCV erkennen kann. Solche Felder werden in der Erkennungsphase vollständig übersprungen, unabhängig von der Qualität der Zuordnungslogik.

Die Muster 1 und 2 treten im Auftragsformular auf, Muster 3 im Bewerbungsformular. Für einfache, standardisierte Formulare mit konventionellem Layout bleibt der geometrische Ansatz die bessere Wahl, da er deutlich schneller ist und keine API-Abhängigkeit hat. Für Formulare, die eines oder mehrere der genannten Muster aufweisen, ist eine semantische Zuordnung notwendig.

4.2 Grundlagen: LLMs für strukturierte Dokumentenanalyse

Large Language Models sind Transformer-basierte neuronale Netze, die auf großen Textmengen vortrainiert wurden.[1] Für die Aufgabe der Label-Feld-Zuordnung sind zwei Eigenschaften dieser Modelle besonders relevant: ihre Fähigkeit zur semantischen Textverarbeitung und ihre Kontextfenster-Architektur.

[1] Vgl. Sahoo et al., 2024.

4.2.1 Semantische Repräsentation von Formularstrukturen

Sprachmodelle repräsentieren Wörter und Sätze als hochdimensionale Vektoren im Einbettungsraum, in dem semantisch ähnliche Ausdrücke räumlich nahe liegen. Das bedeutet, dass Begriffe wie *Telefonnummer*, *Tel.-Nr.* und *Mobilnummer* im Einbettungsraum nahe beieinander liegen, obwohl sie als Zeichenketten vollständig verschieden sind. Für die Formularanalyse ist das ein entscheidender Vorteil: Variationen und Abkürzungen von Feldbezeichnungen werden automatisch als semantisch äquivalent behandelt, ohne dass eine explizite Liste von Synonymen gepflegt werden müsste.
Darüber hinaus haben Sprachmodelle während ihres Trainings Millionen von Dokumenten verarbeitet und dabei implizit Wissen über typische Dokumentstrukturen erworben. Ein Modell weiß, dass *Vorname* und *Nachname* typischerweise zusammen auftreten, dass eine *IBAN* zu einer *BIC* gehört und dass ein Feld neben *Kundennummer:* sehr wahrscheinlich eine Kundennummer enthält. Dieses Kontextwissen steht dem geometrischen Algorithmus nicht zur Verfügung, da er ausschließlich räumliche Relationen bewertet.

4.2.2 Kontextfenster und Prompt-basierte Steuerung

Moderne LLMs verarbeiten alle Eingaben innerhalb eines Kontextfensters, das je nach Modell mehrere tausend bis Millionen Tokens fasst. Ein Token entspricht dabei ungefähr 3–4 Zeichen im Deutschen. Für die Formularanalyse bedeutet das: Der gesamte Prompt, inklusive aller Feldkoordinaten und Textblöcke, muss innerhalb dieses Limits bleiben. Bei typischen einseitigen Formularen mit 10–30 Feldern und 20–50 Textblöcken ist das problemlos erfüllbar.
Die Steuerung des Modellverhaltens erfolgt ausschließlich über den Prompt, welcher die Aufgabe, Daten und gewünschtes Ausgabeformat beschreibt. Diese Prompt-basierte Steuerung ist der einzige Eingriffspunkt, den eine Anwendung gegenüber einem vortrainierten Modell hat, ohne das Modell neu zu trainieren. Das Prompt Engineering ist die Kunst, Prompts so zu formulieren, dass das Modell zuverlässig das gewünschte Ergebnis liefert und ist daher ein zentrales Qualitätsmerkmal der LLM-basierten Zuordnung.[2]

4.2.3 Chain-of-Thought und Reasoning-Modelle

Neuere Modelle wie Gemini 2.5 Flash unterstützen ein internes Reasoning-Verfahren, das unter dem Begriff *Chain-of-Thought* bekannt ist.[3] Das Modell generiert dabei zunächst eine interne Gedankenkette, bevor es die endgültige Antwort ausgibt. Diese Thinking Tokens sind im Output nicht sichtbar, erhöhen aber die Zuverlässigkeit bei komplexen Zuordnungsaufgaben deutlich.
Im gemessenen Beispiel (vgl. Abbildung 4.7) verwendete Gemini 2.5 Flash 4.321 Thinking Tokens neben 1.090 regulären Tokens. Das bedeutet, das Modell hat intern rund viermal so viel Text verarbeitet wie im eigentlichen Prompt-Output sichtbar ist.

4.3 Systemarchitektur der LLM-Integration

Die Einbindung von Sprachmodellen erfolgt als optionale Erweiterung der bestehenden Computer-Vision-Pipeline. Die Felderkennung durch OpenCV, die Textextraktion via PDF-Miner und die OCR-Verarbeitung mit Tesseract bleiben vollständig unverändert. Der LLM-Ansatz greift ausschließlich in der letzten Phase ein: der Zuordnung von Labels zu Feldern.

[2] Vgl. Sahoo et al., 2024.
[3] Vgl. Sahoo et al., 2024.

4.3.1 Einordnung in den Verarbeitungsfluss

Der Verarbeitungsfluss bis zur Zuordnungsphase ist für beide Ansätze identisch. PDFMiner extrahiert zunächst alle Textblöcke aus dem PDF und konvertiert deren Koordinaten vom PDF-Koordinatensystem (Ursprung unten-links) in das OpenCV-Koordinatensystem (Ursprung oben-links). Parallel dazu wird jede PDF-Seite in ein Rasterbild umgewandelt, das nach Vorverarbeitung (Graustufen, Gaussian Blur, Canny Edge Detection) zur Konturdetektion verwendet wird. Die erkannten rechteckigen Konturen werden als potenzielle Eingabefelder klassifiziert, und Textblöcke innerhalb von Feldgrenzen werden als Feldinhalt (nicht als Labels) aus dem Kandidatenpool entfernt.
An diesem Punkt liegen zwei strukturierte Datensätze vor: ein Array erkannter Felder mit Bounding-Boxes und OCR-Text sowie ein Array aller verbleibenden Textblöcke als Label-Kandidaten. Diese Datensätze bilden den Input sowohl für den traditionellen Algorithmus als auch für den LLM-Pfad. Die Routing-Entscheidung zwischen den beiden Pfaden trifft die Funktion `prediction()` basierend auf dem Parameter `use_llm`, den der Nutzer über die Weboberfläche setzt.
Schlägt der LLM-Aufruf fehl (z.B. durch einen API-Timeout, ein Rate-Limit oder einen JSON-Parsing-Fehler in der Antwort), greift automatisch der traditionelle Algorithmus als Fallback ein. Die `llm_details`-Struktur, die während des fehlgeschlagenen LLM-Aufrufs befüllt wurde, bleibt erhalten und enthält den Fehlergrund. Damit ist sichergestellt, dass die Pipeline in jedem Fall ein Ergebnis liefert und Fehler nachvollziehbar dokumentiert werden.

4.3.2 Der LabelFieldAgent als Abstraktionsschicht

Die Klasse `LabelFieldAgent` kapselt die unterschiedlichen API-Formate der drei unterstützten Provider hinter einer einheitlichen Schnittstelle. Aus Sicht der aufrufenden `prediction()`-Funktion ist es unerheblich, welcher Provider verwendet wird: Der Aufruf `agent.match_labels_to_fields(fields, text_layout)` liefert immer dasselbe Rückgabeformat: eine Liste zugeordneter Felder im internen Tupel-Format sowie eine `llm_details`-Struktur mit Metriken.
Diese Kapselung hat zwei Vorteile: Erstens können neue Provider ohne Änderungen an der restlichen Pipeline ergänzt werden. Zweitens können alle Provider unter denselben Bedingungen verglichen werden, da die Eingabedaten und die Auswertungslogik identisch sind.
Unabhängig vom gewählten Provider erfasst der Agent folgende Metriken:

- **Performance:** Verarbeitungszeit in Millisekunden, Token-Verbrauch aufgeteilt nach Input und Output, berechnete API-Kosten sowie Anzahl der Retry-Versuche
- **Ergebnisse:** Anzahl erkannter Felder, Anzahl erfolgreich zugeordneter Felder, Anzahl nicht zugeordneter Felder sowie durchschnittlicher Konfidenzwert
- **Debug-Informationen:** Anzahl verarbeiteter Textblöcke, Prompt-Länge in Zeichen, Validität der JSON-Antwort sowie Finish-Reason des Modells

Diese Metriken werden im Web-Interface als Telemetrie-Dashboard visualisiert (vgl. Abbildung 4.7) und ermöglichen eine direkte Qualitätskontrolle nach jeder Verarbeitung.

4.3.3 Fehlertoleranz und Retry-Mechanismus

Alle drei Provider-Implementierungen sind mit einem dreistufigen Retry-Mechanismus ausgestattet. Bei einem transienten Fehler (z.B. einem HTTP-Fehlercode durch vorübergehende

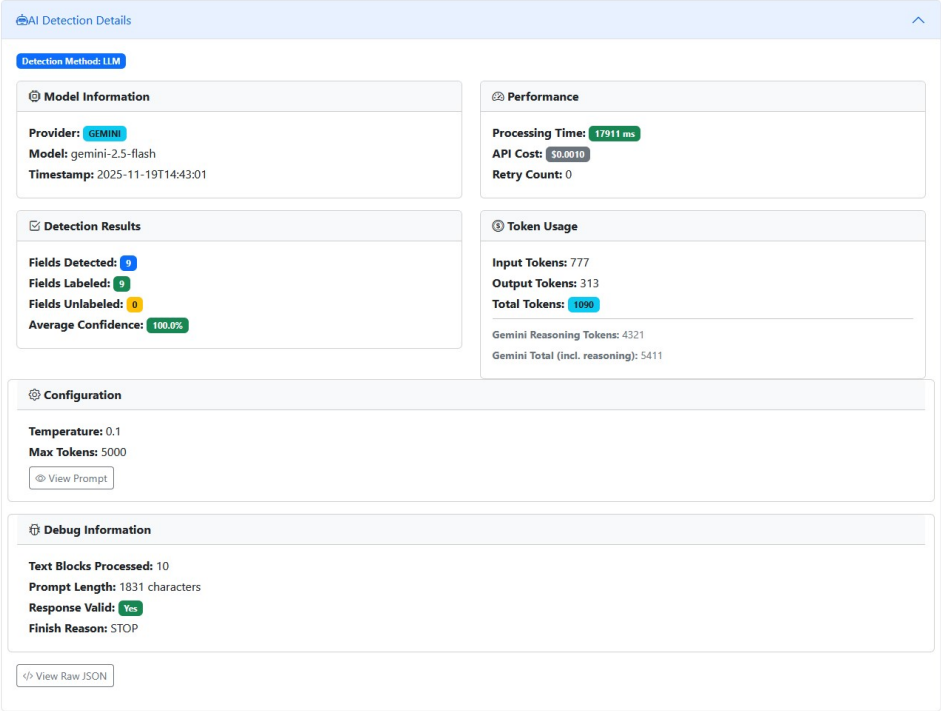

Abbildung 4.7: Telemetrie-Dashboard nach einer LLM-gestützten Verarbeitung mit Gemini 2.5 Flash: 9 von 9 Feldern zugeordnet, durchschnittliche Konfidenz 100 %, Verarbeitungszeit 17.911 ms, Kosten $0,0010, 4.321 interne Reasoning-Tokens

Nichtverfügbarkeit oder einem Rate-Limit) wird der Aufruf bis zu zweimal wiederholt, bevor eine Exception ausgelöst wird. Jeder Retry-Versuch wird in `llm_details[pperformance"][rretry_count"]` dokumentiert.
Ein häufiger Fehlerfall ist das JSON-Parsing: Liefert das Modell keine valide JSON-Antwort (z. B. bei Markdown-Wrappern, die nicht korrekt erkannt wurden, oder bei unvollständiger Ausgabe durch MAX_TOKENS), wird ebenfalls ein Retry ausgelöst. Schlagen alle drei Versuche fehl, gibt `match_labels_to_fields()` den Wert `None` für die Feldliste zurück und setzt `llm_details[rresults"][ffallback_used"]` auf `True`. Die aufrufende `prediction()`-Funktion erkennt den `None`-Rückgabewert und aktiviert den traditionellen Algorithmus.

4.4 Prompt Engineering für Formularanalyse

Das Prompt Engineering ist die zentrale Stellschraube für die Qualität der LLM-basierten Zuordnung. Da das Modell nicht für die spezifische Aufgabe feinabgestimmt wurde, muss der Prompt alle notwendigen Informationen und Anweisungen vollständig enthalten.[4] Gleichzeitig gilt: Je länger der Prompt, desto mehr Tokens werden verbraucht und desto höher sind die Kosten. Ein guter Prompt ist so vollständig wie nötig und so kurz wie möglich.

4.4.1 Struktur des Prompts

Der verwendete Prompt gliedert sich in vier Teile:
Teil 1 – Rollendefinition: Der Prompt beginnt mit *"You are an expert in form analysis."* Diese Formulierung gibt dem Modell einen fachlichen Kontext vor.

[4] Vgl. Sahoo et al., 2024.

Teil 2 – Felddaten: Für jedes erkannte Feld werden Feldindex und Bounding-Box übertragen. Optional wird auch OCR-erkannter Feldinhalt übergeben, falls vorhanden. Das Format ist bewusst kompakt gehalten, um Tokens zu sparen:

```
Field 0: Position: (120, 145) to (380, 175)
Field 1: Position: (786, 145) to (1262, 175)
```

Teil 3 – Textblockdaten: Für jeden Label-Kandidaten werden Text und Bounding-Box übertragen:

```
Text: 'Geb.-Datum:' at position (243, 145) to (469, 175)
Text: 'Tel.-Nr. (mobil):' at position (786, 145) to (1051,
175)
```

Teil 4 – Anweisungsblock: Der Anweisungsblock enthält drei Elemente: erstens Hinweise auf typische Formularlayouts (Labels links, oben oder im Feld), zweitens die Spezifikation des JSON-Ausgabeformats mit den exakten Feldnamen `field_index`, `label` und `confidence`, und drittens die explizite Aufforderung, ausschließlich das JSON-Array ohne erklärenden Begleittext zurückzugeben. Diese letzte Anweisung ist besonders wichtig, da begleitender Text das automatische Parsing zuverlässig zum Scheitern bringt.

4.4.2 Designentscheidungen im Prompt

Mehrere Aspekte des Prompts wurden im Verlauf der Entwicklung iterativ angepasst.
Englisch als Promptsprache: Obwohl die Formulare auf Deutsch sind, ist der Prompt auf Englisch verfasst. LLMs werden typischerweise überwiegend auf englischsprachigen Daten trainiert und zeigen für englischsprachige Anweisungen konsistentere Ergebnisse. Die deutschen Textblöcke werden als Daten übergeben und werden trotzdem korrekt verarbeitet.
Explizite Heuristiken als Hinweise: Der Anweisungsblock listet bekannte Muster auf (Label links, Label oben, Label im Feld). Auch wenn das Modell dieses Wissen implizit aus dem Training mitbringt, verbessert die explizite Nennung die Konsistenz – besonders für Randfälle wie das rechts-stehende Label ← *R.-Datum*.
Konfidenzwert als Qualitätssignal: Die Anforderung eines `confidence`-Wertes zwischen 0 und 100 dient einem doppelten Zweck: Erstens liefert er dem Nutzer im Telemetrie-Dashboard eine Einschätzung der Zuordnungsqualität. Zweitens hat die explizite Anforderung einer Konfidenzschätzung in Experimenten gezeigt, dass sie die Modelle zu sorgfältigerer Analyse veranlasst, da sie ihre Entscheidungssicherheit explizit rechtfertigen müssen.
Temperature-Einstellung: Alle Provider werden mit einer Temperature von 0,1 aufgerufen. Eine Temperature nahe 0 macht das Sampling nahezu deterministisch: Das Modell wählt in jedem Schritt fast immer das wahrscheinlichste nächste Token. Für die Generierung strukturierter JSON-Ausgaben ist das vorteilhaft, da Kreativität hier nicht erwünscht ist.

4.4.3 Token-Verbrauch und Skalierungsverhalten

Für das Auftragsformular aus Abbildung 4.1 ergibt sich ein Prompt mit 1.831 Zeichen. Gemini 2.5 Flash verarbeitete diesen Prompt mit 777 Input-Tokens und erzeugte 313 Output-Tokens (vgl. Abbildung 4.7). Die Prompt-Länge wächst linear mit der Anzahl der Felder und Textblöcke: Jedes Feld trägt etwa 50–70 Zeichen bei, jeder Textblock etwa 60–80 Zeichen.

Für ein Formular mit 30 Feldern und 50 Textblöcken ist mit einem Prompt von ca. 5.500–7.000 Zeichen zu rechnen, was ungefähr 1.800–2.500 Input-Tokens entspricht. Damit bleibt auch dieses Szenario weit innerhalb der Token-Limits der eingesetzten Modelle (GPT-4: 128.000 Tokens, Gemini 2.5 Flash: 1.000.000 Tokens, Ollama/Llama: abhängig vom Modell, typisch 8.000–128.000 Tokens). Die aktuelle Implementierung übergibt daher alle Daten ungefiltert, ohne eine räumliche Vorfilterung der Textblöcke.
Die tatsächliche Prompt-Länge ist für jede Verarbeitung in der Telemetrie als *Prompt Length* einsehbar, was die Identifikation von Dokumenten ermöglicht, die einen ungewöhnlich langen Prompt erzeugen.

4.5 Vergleichende Analyse: Traditionell vs. LLM

Der Mehrwert des LLM-Ansatzes wird anhand der beiden Testformulare aus Abschnitt 4.1 quantifiziert. Tabelle 4.1 stellt beide Ansätze gegenüber.

Tabelle 4.1: Direktvergleich: Traditioneller Algorithmus vs. LLM (Gemini 2.5 Flash) auf beiden Testformularen

Kriterium	**Traditionell**	**Gemini 2.5 Flash**
Auftragsformular (9 Felder)		
Felder korrekt zugeordnet	6 / 9	9 / 9
Erkennungsrate	67 %	100 %
Bewerbungsformular (14 Felder, 2 davon Linienfelder)		
Felder korrekt zugeordnet	9 / 14	12 / 14
Checkboxen korrekt zugeordnet	0 / 3	3/3
Erkennungsrate	64 %	86 %
Linienfelder erkannt	0 / 2	0 / 2
Allgemein		
Verarbeitungszeit	<100 ms	17.911 ms
Kosten pro Aufruf	$0,00	$0,0010

Auf dem Auftragsformular erreicht der traditionelle Algorithmus 67 % (6 von 9 Feldern korrekt zugeordnet), auf dem Bewerbungsformular 64 % (9 von 14). Die Fehler sind in beiden Fällen direkte Folge der in Abschnitt 4.1.3 beschriebenen Layoutmuster. Gemini 2.5 Flash erreicht auf allen erkennbaren Feldern 100 %, benötigt dafür jedoch erheblich mehr Zeit (17.911 ms statt unter 100 ms) und verursacht Kosten von $0,001 pro Verarbeitung.
Bemerkenswert ist, dass die Linienfelder (*Vorname*, *Nachname*) von keinem der beiden Ansätze erkannt werden. Das ist kein Fehler der Zuordnungslogik, sondern ein grundlegendes Limit der vorgelagerten OpenCV-Erkennung: Ohne geschlossene Kontur gibt es für den LLM-Pfad keine Felder, die übergeben werden könnten.

4.5.1 Kostenbetrachtung im Praxiskontext

Die API-Kosten werden anhand provider-spezifischer Preismodelle berechnet, die zwischen Input- und Output-Tokens unterscheiden. Output-Tokens sind teurer, da sie das Ergebnis der eigentlichen Generierungsarbeit des Modells darstellen. Tabelle 4.2 gibt einen Überblick.
Selbst bei 1.000 Formularverarbeitungen pro Monat belaufen sich die Kosten für Gemini 2.5 Flash auf ca. $1,00 pro Monat. Für GPT-4 wären es ca. $15–25,00.

Tabelle 4.2: Kosten- und Zeitvergleich der drei Provider (Schätzwerte für einseitiges Formular mit ca. 10 Feldern)

Provider	Kosten/Aufruf	Verarbeitungszeit	Token-Tracking
OpenAI GPT-4	ca. $0,015–0,025	ca. 3.000–6.000 ms	vollständig
Gemini 2.5 Flash	ca. $0,001	ca. 10.000–20.000 ms	inkl. Thinking Tokens
Ollama (lokal)	$0,00	hardware-abhängig	nicht verfügbar

Diese Größenordnung ist für die meisten Unternehmensanwendungen vernachlässigbar. Für Szenarien mit sehr hohem Volumen, ohne zuverlässige Internetverbindung oder hohen Datenschutzanforderungen ist Ollama die sinnvollere Wahl.

4.5.2 Bewertung der Genauigkeitsunterschiede

Die gemessenen Erkennungsraten von 67 % bzw. 64 % für den traditionellen Ansatz sind auf die bewusst schwierigen Testformulare zurückzuführen, die gezielt Layoutmuster enthalten, bei denen der Algorithmus versagt. Bei einfachen, standardisierten Formularen erreicht der traditionelle Algorithmus vergleichbare oder identische Ergebnisse wie das LLM, bei einem Bruchteil der Verarbeitungszeit. Der LLM-Ansatz entfaltet seinen Vorteil gezielt bei Formularen, die von der Standardstruktur abweichen.
Dies motiviert die Designentscheidung, beide Ansätze als gleichwertige, wählbare Optionen zu implementieren: Der Nutzer kann situationsabhängig entscheiden, ob Geschwindigkeit (traditionell) oder maximale Erkennungsqualität (LLM) prioritär ist. Die Fallback-Logik stellt dabei sicher, dass ein fehlgeschlagener LLM-Aufruf immer ein Ergebnis liefert.

4.6 Robustheit und Grenzen des LLM-Ansatzes

Trotz der Vorteile bei komplexen Layouts hat der LLM-Ansatz eigene Einschränkungen, die für den praktischen Einsatz relevant sind und in der Implementierung berücksichtigt werden mussten.

4.6.1 Nicht-Determinismus

Sprachmodelle sind im Kern probabilistische Systeme: Bei identischem Prompt können bei wiederholten Aufrufen leicht unterschiedliche Zuordnungen entstehen, selbst bei einer Temperature von 0,1. Das ist ein grundlegender Unterschied zum traditionellen Algorithmus, der für denselben Input immer dasselbe Ergebnis liefert. Für die meisten Formularverarbeitungsszenarien ist dieser Nicht-Determinismus akzeptabel, da die Varianz bei niedrigen Temperature-Werten gering ist. Für sicherheitskritische Anwendungen, die vollständige Reproduzierbarkeit erfordern, ist der LLM-Ansatz ohne zusätzliche Maßnahmen (z. B. festes Seed-Setting, sofern vom Provider unterstützt) ungeeignet.

4.6.2 API-Abhängigkeit und Datenschutz

Die Cloud-basierten Provider OpenAI und Gemini erfordern eine stabile Internetverbindung und sind von der Verfügbarkeit externer Dienste abhängig. API-Ausfälle oder Rate-Limits können die Verarbeitung verzögern. Außerdem verlassen die Formulardaten inklusive aller erkannten Textblöcke und Feldkoordinaten das lokale System und werden an externe Server übertragen. Für Formulare mit vertraulichem Inhalt (z. B. Personalbögen, Finanzformulare) ist das je nach Datenschutzanforderung problematisch.

Ollama löst das Datenschutzproblem durch vollständig lokale Ausführung: Alle Daten bleiben auf dem System. Die Einschränkungen sind jedoch die fehlenden Token-Statistiken und die hardware-abhängige Leistung. Auf Systemen ohne dedizierte GPU sind Open-Source-Modelle in der Regel deutlich langsamer und weniger präzise als die Cloud-Varianten.

4.6.3 Qualität der JSON-Ausgabe

Die zuverlässige Generierung validen JSONs ist eine bekannte Herausforderung bei LLMs. Trotz expliziter Formatanweisungen im Prompt kann es vorkommen, dass Modelle strukturelle Fehler einbauen (fehlende Klammern, abgeschnittene Arrays) oder ihr JSON in Markdown-Codeblöcke einbetten. Die Implementierung behandelt beide Fälle: Markdown-Wrapper werden systematisch entfernt, und abgeschnittene Antworten (erkennbar am Finish-Reason `MAX_TOKENS`) lösen einen Retry aus.
Eine robustere Lösung wäre der Einsatz von *Structured Outputs* (eine API-Funktion, die von OpenAI und Gemini mittlerweile unterstützt wird), welche die Ausgabe des Modells direkt gegen ein JSON-Schema validiert. Diese Funktion wurde in der aktuellen Implementierung noch nicht eingesetzt, wäre aber eine sinnvolle Erweiterung für eine produktive Version.

4.6.4 Skalierbarkeit bei mehrseitigen Dokumenten

Die aktuelle Implementierung verarbeitet alle Felder und Textblöcke eines Dokuments in einem einzigen LLM-Aufruf. Bei mehrseitigen Dokumenten mit vielen Feldern kann die resultierende Prompt-Länge die Token-Limits kleinerer Modelle überschreiten. Selbst wenn die Limits nicht überschritten werden, steigen Kosten und Verarbeitungszeit proportional mit der Dokumentgröße.
Für zukünftige Versionen wäre eine seitenweise Verarbeitung sinnvoll: Jede Seite wird als eigenständiges Zuordnungsproblem behandelt, was die Prompt-Länge konstant klein hält und die parallele Verarbeitung mehrerer Seiten ermöglicht. Alternativ könnte eine räumliche Vorfilterung implementiert werden, die für jedes Feld nur Textblöcke innerhalb eines bestimmten Radius als Label-Kandidaten einbezieht. Dies würde die Prompt-Länge wesentlich reduzieren.

5 Web-Interface und Management-System

Die in den vorangegangenen Kapiteln beschriebenen Algorithmen zur Felderkennung und semantischen Klassifikation sind technische Komponenten, die für sich allein noch keinen praktischen Mehrwert für Endnutzer bieten. Erst die Einbettung in eine zugängliche Benutzeroberfläche macht das System nutzbar: Ein Nutzer soll ein bestehendes PDF-Formular hochladen, Felder automatisch erkennen lassen, die Ergebnisse prüfen und korrigieren, und schließlich wiederholt befüllte PDFs aus diesem Template erzeugen können.
Dieses Kapitel beschreibt das entwickelte Web-Interface *OpenPDF* und das zugehörige Management-System. Es gliedert sich wie folgt: Abschnitt 5.1 erläutert die Architekturentscheidungen. Abschnitt 5.2 beschreibt die Template-Verwaltung von der Erstellung bis zur Bearbeitung. Abschnitt 5.3 behandelt den Prozess der PDF-Generierung. Abschnitt 5.4 erklärt das Dataset-System zur Wiederverwendung von Eingabedaten. Abschnitt 5.5 beschreibt die Verlaufsverfolgung. Abschnitt 5.6 zeigt anhand eines durchgehenden Beispiels den vollständigen Arbeitsablauf vom Formular-Upload bis zur fertigen PDF-Ausgabe.

5.1 Architektur und technische Grundlagen

Das System wurde als Webanwendung implementiert, um plattformunabhängigen Zugriff über den Browser zu ermöglichen. Es sind keine lokalen Installationen auf dem Endgerät notwendig, und die Anwendung ist über jeden modernen Browser erreichbar. Als Framework wurde Django 5.0 gewählt.[1]

5.1.1 Warum Django?

Django ist ein Python-basiertes Webframework: Es liefert alle wesentlichen Komponenten einer Webanwendung direkt mit, ohne externe Abhängigkeiten für Kernfunktionalität zu erfordern. Für dieses Projekt sind besonders drei Django-Komponenten relevant.
Das integrierte Object-Relational Mapping (ORM) erlaubt es, Datenbankoperationen in Python zu formulieren, ohne direkte SQL-Abfragen zu schreiben. Datenbankeinträge werden als Python-Objekte behandelt, was Entwicklungsgeschwindigkeit und Lesbarkeit des Codes erhöht. Als Datenbank kommt SQLite zum Einsatz, das als dateibasiertes System ohne separate Serverinstanz auskommt und für den Prototyp-Einsatz geeignet ist.
Die Template-Engine von Django trennt Präsentation und Logik: HTML-Dateien enthalten Platzhalter, die beim Rendern durch tatsächliche Daten ersetzt werden. Das ermöglicht es, dieselben View-Funktionen für unterschiedliche Darstellungen (z. B. Desktop- und Mobile-Ansicht) zu verwenden.
Das eingebaute Authentifizierungssystem verwaltet Benutzerkonten, Sitzungen und Zugriffsrechte. Im aktuellen System wird der Admin-Bereich von Django genutzt, um direkt auf alle Datenbankeinträge zuzugreifen.
Um das System in einem Unternehmen nutzbar zu machen, wäre es sinnvoll für jeden Mitarbeiter auch einen Nutzer anzulegen um nachzuvollziehen wer welchen Auftrag wann er-

[1] Vgl. Django Software Foundation, 2024.

stellt hat. Aus Datenschutzgründen kann es auch nötig sein ein Rechteverwaltungssystem zu implementieren um den Nutzern nur einen eingeschränkten Zugriff auf Templates und Datasets zu geben, da diese sensitive Kundendaten enthalten können.

5.1.2 MVT-Architekturmuster

Django folgt dem Model-View-Template (MVT) Muster.[2] Das Modell (`models.py`) definiert die Datenbankstruktur: welche Entitäten existieren, welche Felder sie haben und wie sie miteinander verknüpft sind. Die View (`views.py`) enthält die Anwendungslogik: Sie empfängt HTTP-Anfragen, ruft Modell-Operationen auf und gibt Antworten zurück. Das Template (HTML-Dateien) übernimmt ausschließlich die Darstellung der Daten im Browser.
Diese Trennung hat einen praktischen Vorteil für das vorliegende System: Die Algorithmen zur Felderkennung (Kapitel 3) und die LLM-Anbindung (Kapitel 4) sind vollständig von der Web-Schicht entkoppelt. Sie werden von den Views als Python-Module importiert und aufgerufen – Änderungen an der Oberfläche berühren die Algorithmen nicht und umgekehrt.

5.1.3 Navigation und Oberflächenstruktur

Die Navigationsleiste gliedert das System in drei Hauptbereiche: *Templates* mit Untermenü für manuelle Erstellung und automatisierte Generierung, *Datasets* mit der Übersicht aller gespeicherten Datensätze, und *History* mit einer Liste aller erstellten Templates. Rechts befindet sich der Zugang zum Admin-Bereich, der aktuell einzige Weg um Templates wieder zu löschen.

5.2 Template-Verwaltung

Die Template-Verwaltung umfasst den gesamten Lebenszyklus eines PDF-Templates. Er beginnt bei der initialen Erstellung, geht über die automatisierte Felderkennung und bis hin zur manuellen Korrektur. Sie ist der funktionale Kern des Systems, da alle weiteren Aktionen (PDF-Generierung, Dataset-Nutzung) auf einem fertig konfigurierten Template aufbauen.

5.2.1 Template-Übersicht

Die Template-Übersicht (Abbildung 5.1) ist die Startseite des Systems und zeigt alle angelegten Templates als Kacheln. Jede Kachel enthält den Template-Namen, eine Miniaturvorschau des Quelldokuments sowie vier direkte Aktionstasten.
Details öffnet die detailansicht mit allen Template-Metadaten, den erkannten Feldern und den LLM-Telemetriedaten.
Edit öffnet die Bearbeitungsmaske, in der alle Felder des Templates angepasst werden können. Das umfasst Feldnamen, Positionen, Typen und Formatierung.
New PDF öffnet das Eingabeformular zur Generierung einer neuen PDF aus diesem Template. Die erkannten Feldnamen dienen dabei als Beschriftungen der Eingabefelder.
Preload with Dataset öffnet ein Dropdown mit allen verfügbaren Datasets und befüllt das Eingabeformular mit den Werten des gewählten Datasets vor. Dis ist ein wertvolles Feature für Nutzer, die regelmäßig Dokumente mit ähnlichen Daten erstellen.
Die zwei Schaltflächen oben im Seitenbereich erlauben das Anlegen neuer Templates: manuell über *New Template* oder automatisiert über *Generate Template*.

[2] Vgl. Django Software Foundation, 2024.

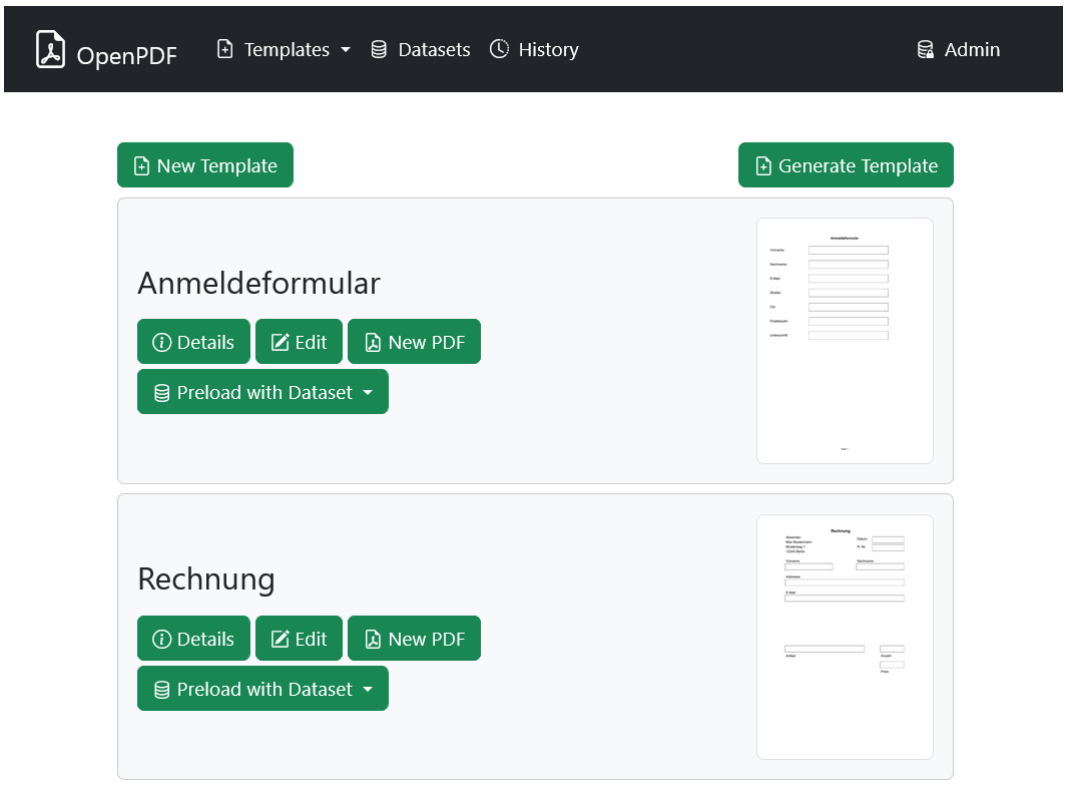

Abbildung 5.1: Template-Übersicht: Alle angelegten Templates mit Vorschaubild und direkten Aktionen. Links *New Template* für manuelle Erstellung, rechts *Generate Template* für automatisierte Generierung.

5.2.2 Automatisierte Template-Generierung

Die automatisierte Template-Generierung (Abbildung 5.2) ist der Einstiegspunkt für die Computer-Vision-Pipeline. Der Nutzer gibt einen Template-Namen an, wählt eine PDF-Quelldatei aus und entscheidet sich für eine von zwei Erkennungsmethoden.

Traditional Algorithm verwendet den in Kapitel 3 beschriebenen geometrischen Ansatz. Er arbeitet regelbasiert, benötigt keine externe API und liefert Ergebnisse in unter einer Sekunde. Er ist die richtige Wahl für Formulare mit konventionellem Layout.

AI-Powered Detection verwendet die in Kapitel 4 beschriebene LLM-basierte Zuordnung. Bei Auswahl dieser Option erscheint ein erweiterter Bereich mit drei Provideroptionen: OpenAI GPT-4, Google Gemini und Local Ollama.

Ein besonderes Feature ist die *Check API Connection*-Schaltfläche. Sie ruft den Endpunkt `/api/check-llm-status/` auf, der die Erreichbarkeit des gewählten Providers und die Gültigkeit des konfigurierten API-Schlüssels prüft. Das Ergebnis wird unmittelbar neben der Schaltfläche angezeigt – im Bild als grünes *Connected*. So erfährt der Nutzer bereits vor dem Hochladen der Datei, ob die KI-Erkennung verfügbar ist, und muss nicht erst nach einem langen Upload-Prozess eine Fehlermeldung erhalten.

Nach dem Klick auf *Generate Template* wird die PDF an die Backend-Pipeline übergeben. Die Verarbeitung umfasst: Konvertierung der PDF-Seiten in Rasterbilder, Textextraktion durch PDFMiner, Vorverarbeitung und Kantendetektion durch OpenCV, Konturerkennung zur Feldidentifikation, OCR-basierte Texterkennung in den Feldern und schließlich die Label-Feld-Zuordnung durch den gewählten Algorithmus. Bei einem Fehler im LLM-Pfad greift die Fallback-Logik automatisch auf den traditionellen Algorithmus zurück.

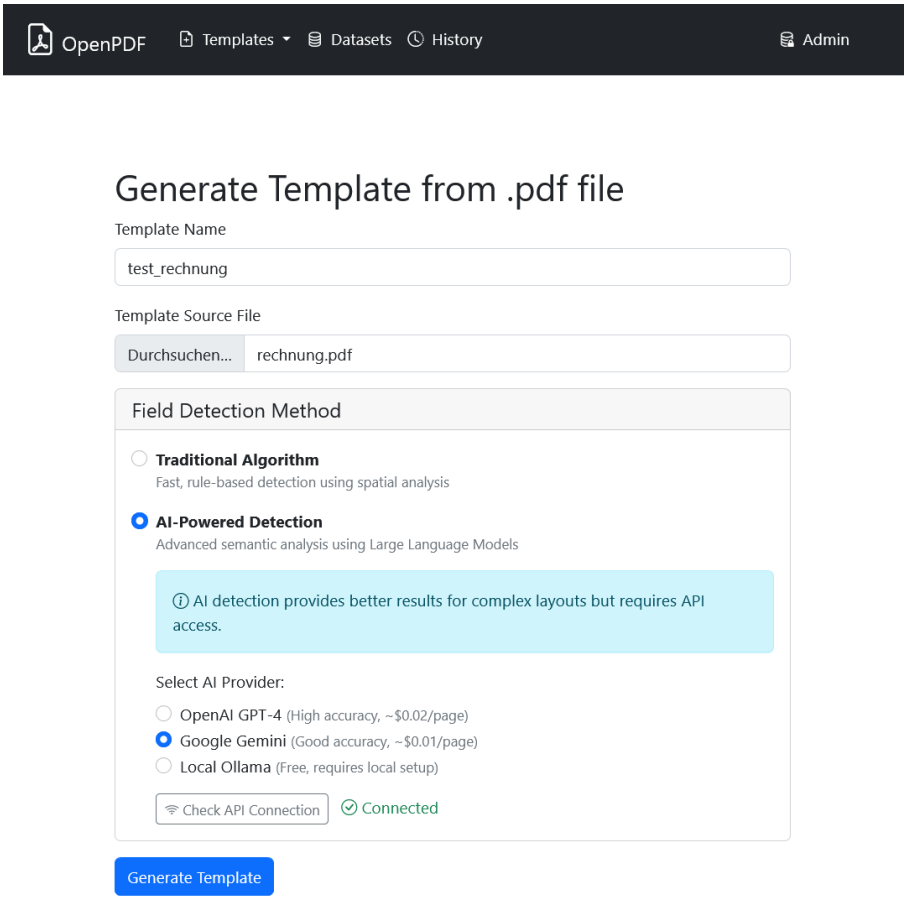

Abbildung 5.2: Formular zur automatisierten Template-Generierung: Name, Quelldatei und Erkennungsmethode. Bei Auswahl von *AI-Powered Detection* erscheint die Providerauswahl und eine Live-Verbindungsprüfung.

5.2.3 Template-Detailansicht

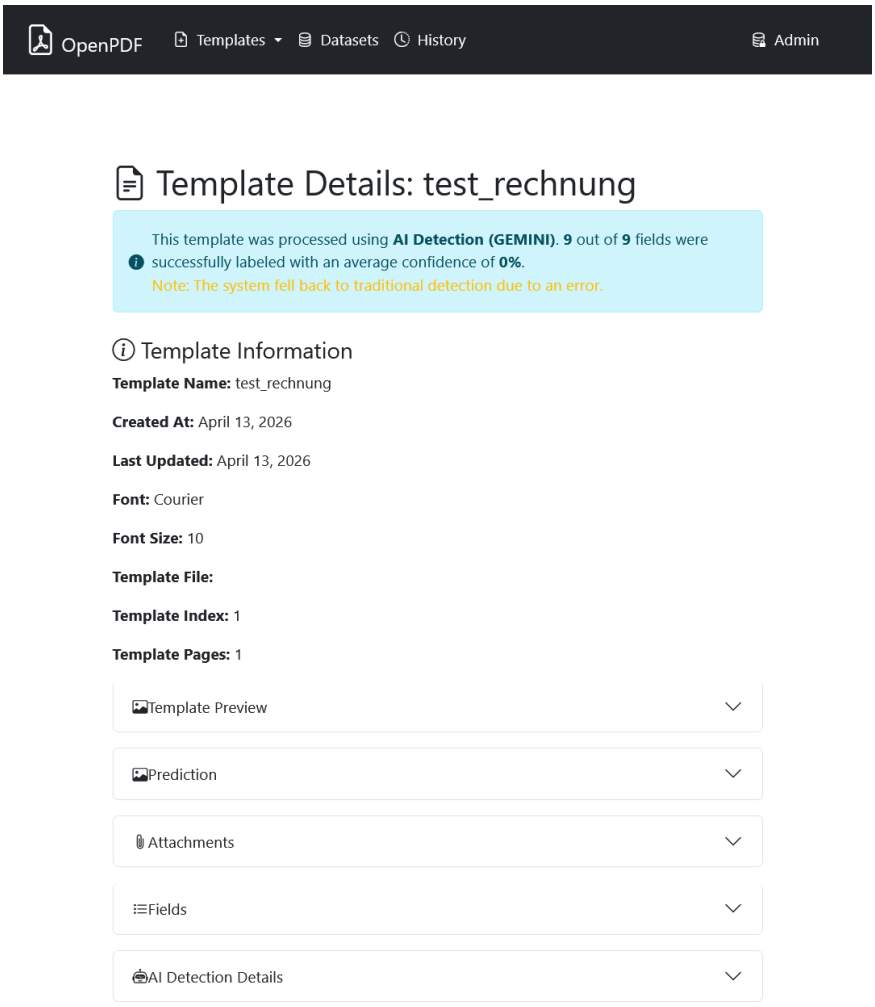

Abbildung 5.3: Template-Detailansicht nach automatisierter Erkennung: Statusmeldung über verwendete Methode und Erkennungsqualität, Metadaten sowie aufklappbare Sektionen für Vorschau, Prediction-Bild, Anhänge, Felder und KI-Telemetrie.

Nach der Generierung (oder über den *Details*-Button aus der Übersicht) zeigt die Detailansicht (Abbildung 5.3) alle Informationen zu einem Template.
Am oberen Rand erscheint eine farbige Statusleiste, die über die verwendete Erkennungsmethode informiert. Im gezeigten Beispiel wurde *AI Detection (GEMINI)* verwendet, wobei 9 von 9 Feldern erfolgreich zugeordnet wurden. Zusätzlich ist ein Hinweis sichtbar, dass das System auf traditionelle Erkennung zurückgefallen ist – die Anzeige der ursprünglich angeforderten Methode und des tatsächlich verwendeten Fallbacks hilft beim Nachvollziehen des Systemverhaltens.
Die Metadaten-Sektion zeigt Template-Name, Erstellungs- und Änderungsdatum, Schriftart und -größe sowie Seitenanzahl. Darunter folgen fünf aufklappbare Sektionen:
Template Preview zeigt das Originaldokument als Bild zur Orientierung.
Prediction zeigt das von OpenCV erzeugte Erkennungsbild mit eingezeichneten Bounding-Boxes der erkannten Felder und den zugeordneten Labels. Dieser Abschnitt ist der wichtigste Qualitätskontrollpunkt nach einer automatisierten Erkennung: Fehlerhafte Zuordnungen oder fehlende Felder sind hier sofort sichtbar.
Attachments listet alle angehängten PDF-Dokumente mit ihren Indizes.

Fields zeigt alle erkannten und gespeicherten Felder mit Namen, Typ, Position und Seitenindex.
AI Detection Details zeigt das Telemetrie-Dashboard (vgl. Kapitel 4, Abbildung 4.7) mit Verarbeitungszeit, Token-Verbrauch, Kosten und Konfidenzwerten – allerdings nur wenn die KI-Erkennung verwendet wurde.

5.2.4 Template-Bearbeitung

Nach der automatisierten Erkennung können alle Felder eines Templates über die Bearbeitungsmaske angepasst werden. Diese Korrekturmöglichkeit ist aus zwei Gründen wichtig: Erstens liefern die Erkennungsalgorithmen selten perfekte Ergebnisse bei ungewöhnlichen Layouts. Zweitens ändern sich Formulare im Laufe der Zeit, und Templates müssen dann nachgepflegt werden.
In der Bearbeitungsmaske wird jedes Feld als editierbare Zeile dargestellt, mit Feldern für Name, Typ (Text, Textarea, Checkbox), Seitenindex, X/Y-Koordinaten und Formatierung. Felder können einzeln gelöscht oder neue Felder manuell hinzugefügt werden. Beim Speichern werden alle Felder des Templates durch die neu definierten ersetzt – ein einfaches, aber effektives Vorgehen, das keine Konfliktauflösung zwischen alten und neuen Daten erfordert.
Bei der manuellen Template-Erstellung (über *New Template*) entfällt die automatisierte Erkennungsphase. Der Nutzer definiert alle Felder von Anfang an selbst und nutzt die Template-Vorschau als visuelle Orientierung für die Koordinateneingabe.

5.3 PDF-Generierung

Die PDF-Generierung ist die eigentliche Kernfunktion des Systems aus Nutzersicht: Ein Template wird mit konkreten Daten befüllt und als fertige PDF heruntergeladen.

5.3.1 Eingabeformular

Das Eingabeformular (Abbildung 5.4) wird dynamisch aus der Feldstruktur des jeweiligen Templates generiert. Jedes Feld des Templates erscheint als beschriftetes Eingabefeld, wobei der Feldname als Label dient.
Am unteren Rand des Formulars befinden sich drei Elemente: Eine Attachment-Checkbox, die es dem Nutzer erlaubt zu steuern, ob optionale Anhänge in die generierte PDF eingebunden werden sollen. Eine Dataset-Auswahl-Schaltfläche, die das Formular mit einem gespeicherten Datensatz vorausfüllt. Und ein Dateinamen-Feld, das den Namen der heruntergeladenen PDF-Datei bestimmt.
Das Formular unterstützt auch den *Preload*-Modus: Wenn der Nutzer aus der History-Ansicht auf *Preload* klickt, wird die URL mit einer Generations-ID erweitert (`/new/<temp_id>/<gen_id>/`), und das Formular wird automatisch mit den Werten dieser früheren Generierung befüllt. Das ist nützlich, wenn ein Dokument mit leicht geänderten Daten erneut erstellt werden soll, ohne alle Felder von Grund auf neu einzugeben.

5.3.2 Technische Verarbeitung bei der Generierung

Nach dem Absenden des Formulars übernimmt die Generierungslogik die Verarbeitung. Sie liest das Template und alle zugehörigen Felder aus der Datenbank, erstellt für jede Seite des Templates einen ReportLab-Canvas und rendert die eingegebenen Textwerte an den gespeicherten Koordinaten.

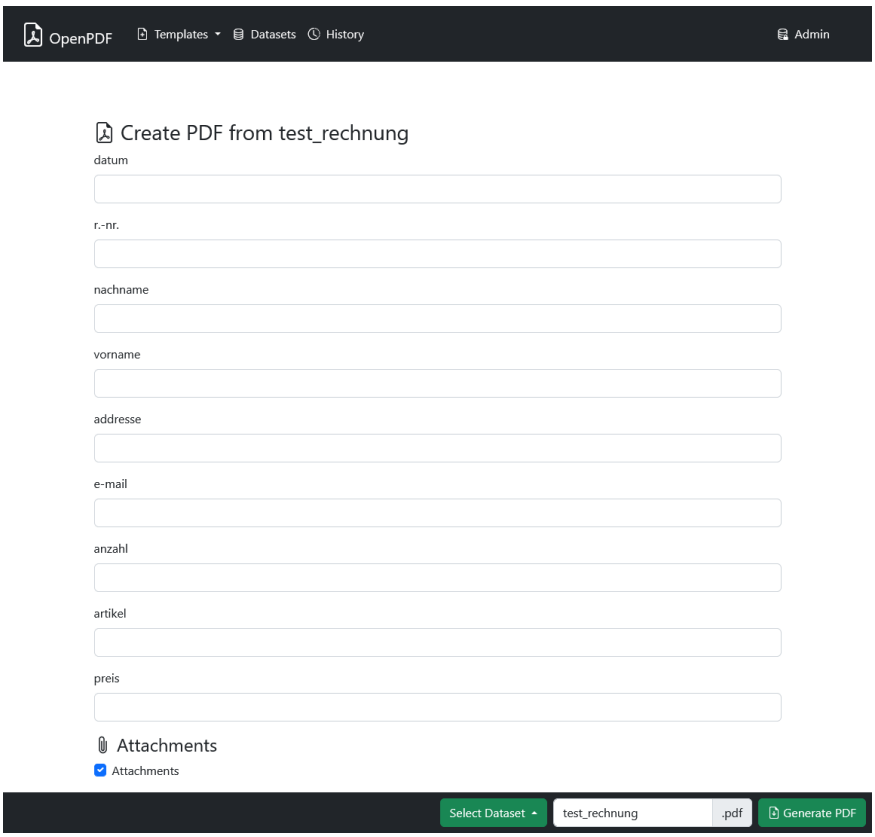

Abbildung 5.4: Eingabeformular zur PDF-Generierung aus dem Template *test_rechnung*: Die erkannten Feldnamen dienen als Beschriftungen. Unten: Attachment-Checkbox, Dataset-Auswahl und Dateiname.

Das System unterscheidet drei Feldtypen bei der Darstellung: Einzeilige Textfelder werden direkt mit der angegebenen Schriftart und -größe an der definierten Position gerendert. Mehrzeilige Textbereiche (Textarea) unterstützen automatische Zeilenumbrüche und werden über den ReportLab-Textmechanismus gesetzt. Checkboxen werden durch ein einfaches 'X' an der Feldposition dargestellt.
Nach dem Rendering der beschrifteten Ebene wird diese mit dem Original-PDF überlagert: Die ReportLab-Seiten werden als transparente Ebene auf die entsprechenden Seiten des Original-PDFs gelegt und Anhänge werden entsprechend ihres Index eingeordnet.
Das fertige Dokument wird direkt als Download-Antwort zurückgegeben und gleichzeitig als *Generation*-Eintrag in der Datenbank gespeichert, inklusive der verwendeten Feldwerte.

5.4 Dataset-Management

Das Dataset-System adressiert ein praktisches Problem bei der wiederholten Nutzung: Wenn Nutzer regelmäßig ähnliche Dokumente erstellen (z.B. monatliche Rechnungen für denselben Kunden) müssen dieselben Daten (Name, Adresse, E-Mail) immer wieder neu eingegeben werden. Datasets speichern diese Daten einmalig und machen sie für beliebige Generierungen wiederverwendbar.

5.4.1 Dataset-Erstellung

Datasets können auf zwei Wegen erstellt werden: Aus der History-Ansicht nach einer Generierung (über *New Dataset*), wobei die Feldwerte der abgeschlossenen Generierung auto-

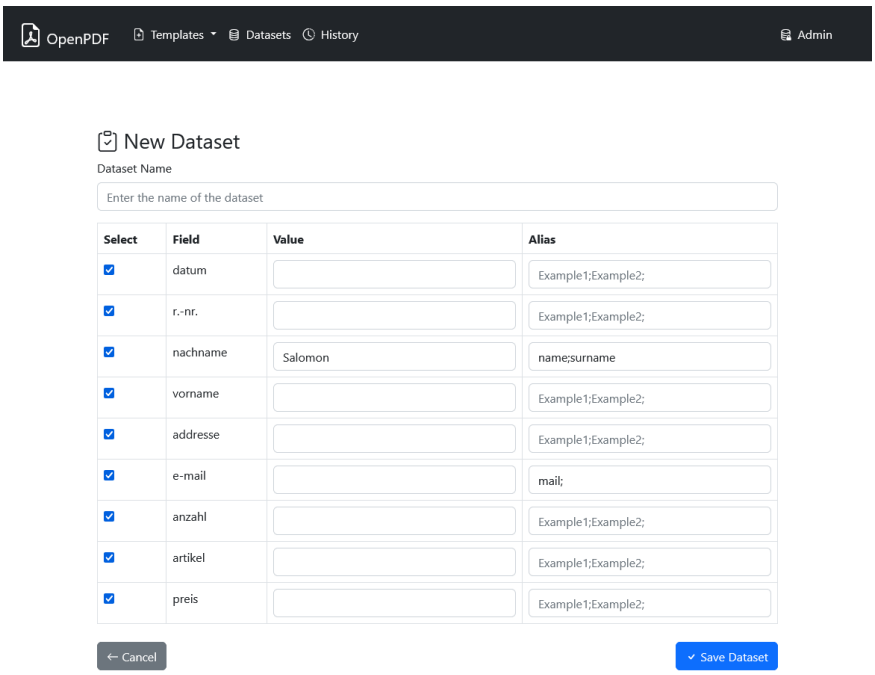

Abbildung 5.5: Dataset-Erstellung: Tabellarische Eingabe von Feldwerten und Aliases. Im Beispiel sind *nachname* mit dem Wert *Salomon* und den Aliases *name;surname* sowie *e-mail* mit dem Alias *mail* befüllt.

matisch vorausgefüllt werden. Oder manuell über *New Dataset* im Datasets-Bereich.
Das Erstellungsformular (Abbildung 5.5) zeigt alle Felder des zugehörigen Templates in einer dreispaltigen Tabelle. Die erste Spalte enthält Checkboxen zum selektiven Einschließen von Feldern, es muss aber nicht jedes Feld in einem Dataset gespeichert werden. Die zweite Spalte zeigt den Feldnamen. Die dritte Spalte nimmt den zu speichernden Wert auf. Die vierte Spalte ermöglicht die Eingabe von Aliases.
Das Alias-System ist ein wichtiges Feature für die Flexibilität des Dataset-Konzepts. Ein Alias ist ein alternativer Name, unter dem der Feldwert beim Anwenden des Datasets gesucht wird. Im Beispiel in Abbildung 5.5 hat das Feld *nachname* den Wert *Salomon* und die Aliases *name;surname*. Das bedeutet: Wenn dieses Dataset auf ein Template angewendet wird, das kein Feld namens *nachname* hat, aber ein Feld namens *name* oder *surname*, wird der Wert *Salomon* trotzdem korrekt zugeordnet. Das macht Datasets über Templategrenzen hinweg nutzbar, ohne für jede Feldnamenvariante ein separates Dataset anlegen zu müssen. Mehrere Aliases werden durch Semikolon getrennt eingegeben (`name;surname`), wie in der Placeholder-Beschriftung *Example1;Example2;* angedeutet.

5.4.2 Dataset-Übersicht und Anwendung

Die Dataset-Übersicht (Abbildung 5.6) listet alle gespeicherten Datasets mit ID, Name, Quelltemplate, Feldanzahl und Erstellungsdatum. Die Spalte *Source Template* zeigt, aus welchem Template das Dataset ursprünglich erstellt wurde.
Die *Fill Template*-Schaltfläche öffnet ein Dropdown mit allen verfügbaren Templates. Im Bild ist das Dropdown für Dataset 3 (*salo*) aufgeklappt und zeigt eine Liste aller Templates. Der Nutzer kann also das Dataset *salo* direkt auf *Anmeldeformular* oder irgendein anderes Template anwenden. Das System überträgt dabei alle Felder, deren Name oder Alias im Zieltemplate vorkommt, und öffnet das Generierungsformular mit diesen Werten vorausgefüllt.

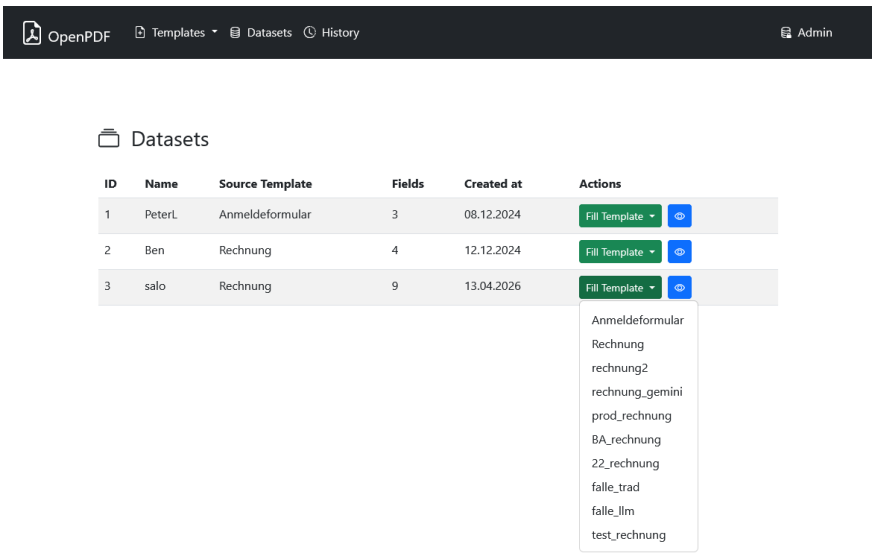

Abbildung 5.6: Dataset-Übersicht: Drei gespeicherte Datensätze mit Quelltemplates und Feldanzahl. Über *Fill Template* mit Dropdown wird ein Dataset direkt auf ein beliebiges Template angewendet.

Das Auge-Icon rechts neben *Fill Template* öffnet eine Detailansicht des Datasets, in der alle gespeicherten Werte und Aliases eingesehen und bearbeitet werden können.

5.5 Generierungshistorie

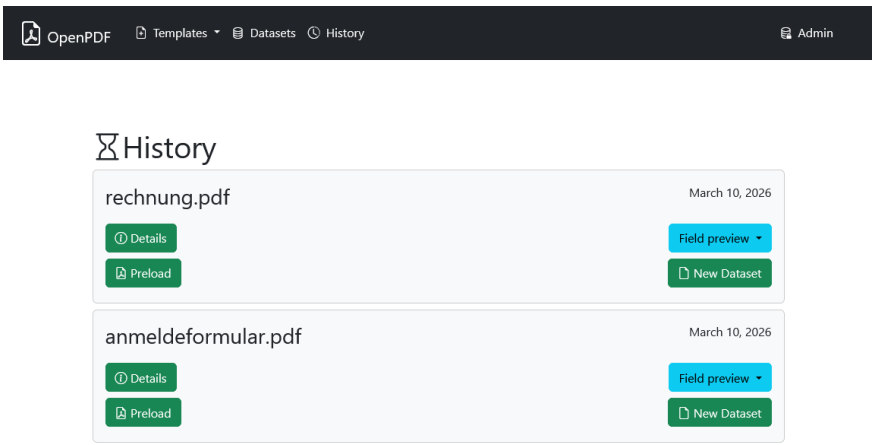

Abbildung 5.7: History-Ansicht: Chronologische Liste aller PDF-Generierungen mit Dateinamen, Datum und Direktaktionen für Details, Preload, Field Preview und Dataset-Erstellung.

Die History-Ansicht (Abbildung 5.7) protokolliert jede durchgeführte PDF-Generierung chronologisch. Jeder Eintrag zeigt den Dateinamen der erzeugten PDF und das Erstellungsdatum.
Vier Aktionen stehen für jeden Eintrag zur Verfügung:
Details öffnet eine Detailseite der Generierung mit allen verwendeten Feldwerten und dem zugehörigen Template.
Preload öffnet das Generierungsformular des zugehörigen Templates und befüllt es mit den Werten dieser Generierung vor. Das ist der schnellste Weg, ein Dokument mit leicht geänderten Daten erneut zu erstellen: statt alle Felder neu einzugeben, werden die Werte

der letzten Generierung übernommen und nur die geänderten Felder angepasst.
Field Preview öffnet ein Dropdown, das zeigt, welche Felder bei der Generierung welche Werte hatten. Dies bietet eine schnelle visuelle Kontrolle der verwendeten Eingaben ohne die Detailseite zu öffnen.
New Dataset leitet zur Dataset-Erstellung weiter und befüllt das Formular mit den Werten dieser Generierung vor. Das ist der typische Workflow, um aus einer einmaligen Generierung einen wiederverwendbaren Datensatz zu erstellen.

5.6 Beispieldurchlauf: Von der Quelldatei zur befüllten PDF

Um das Zusammenspiel aller Komponenten zu veranschaulichen, wird in diesem Abschnitt ein vollständiger Arbeitsablauf am Beispiel des Rechnungsformulars *test_rechnung* beschrieben. Der Durchlauf umfasst alle Hauptfunktionen des Systems: Template-Generierung, PDF-Erstellung, Verlaufsprotokoll, Dataset-Erstellung und Dataset-Anwendung.

5.6.1 Schritt 1: Template automatisch generieren

Ausgangspunkt ist eine bestehende PDF-Datei *rechnung.pdf*, die als leeres Formular vorliegt und wiederholt befüllt werden soll. Über *Generate Template* in der Navigationsleiste wird das Generierungsformular geöffnet (vgl. Abbildung 5.2).
Der Nutzer gibt den Template-Namen *test_rechnung* ein und wählt *rechnung.pdf* als Quelldatei. Als Erkennungsmethode wird *AI-Powered Detection* mit Provider *Google Gemini* gewählt. Über *Check API Connection* wird sichergestellt, dass die Gemini-API erreichbar ist (*Connected*). Ein Klick auf *Generate Template* startet die Verarbeitungspipeline.
Das System konvertiert die PDF in ein Rasterbild, extrahiert alle Textblöcke per PDFMiner, erkennt Feldkonturen per OpenCV und übergibt die aufbereiteten Daten an Gemini 2.5 Flash. Das Modell ordnet alle 9 Felder korrekt zu (vgl. Kapitel 4). Das Template wird in der Datenbank gespeichert und der Nutzer wird zur Detailansicht weitergeleitet.

5.6.2 Schritt 2: Ergebnis prüfen und ggf. korrigieren

In der Detailansicht (vgl. Abbildung 5.3) sieht der Nutzer die Statusmeldung: *This template was processed using AI Detection (GEMINI). 9 out of 9 fields were successfully labeled.* Im aufgeklappten *Prediction*-Abschnitt sind die erkannten Felder eingezeichnet.
Entspricht das Ergebnis den Erwartungen, kann direkt mit der PDF-Generierung fortgefahren werden. Falls Felder fehlen oder falsch benannt sind, führt ein Klick auf *Edit* zur Bearbeitungsmaske, in der Korrekturen vorgenommen werden können.

5.6.3 Schritt 3: Erste PDF generieren

Über den *New PDF*-Button in der Template-Übersicht (vgl. Abbildung 5.1) wird das Eingabeformular geöffnet (vgl. Abbildung 5.4). Die neun Felder des Templates erscheinen als Eingabefelder: *datum*, *r.-nr.*, *nachname*, *vorname*, *addresse*, *e-mail*, *anzahl*, *artikel* und *preis*. Der Nutzer füllt alle Felder aus, belässt den vorgeschlagenen Dateinamen *test_rechnung.pdf* und klickt auf *Generate PDF*. Die fertige PDF wird heruntergeladen.

5.6.4 Schritt 4: Dataset aus der Generierung erstellen

In der History-Ansicht (vgl. Abbildung 5.7) erscheint die soeben erstellte *rechnung.pdf*. Über *New Dataset* wird die Dataset-Erstellung geöffnet (vgl. Abbildung 5.5). Das Formular ist bereits mit den Werten der Generierung vorausgefüllt.
Der Nutzer vergibt den Dataset-Namen *salo* und trägt für das Feld *nachname* den Alias *name;surname* ein, so kann das Dataset auch auf Templates mit abweichender Feldbenennung angewendet werden. Ein Klick auf *Save Dataset* speichert den Datensatz.

5.6.5 Schritt 5: Dataset für weitere Generierungen nutzen

In der Dataset-Übersicht (vgl. Abbildung 5.6) ist das neue Dataset *salo* sichtbar. Beim nächsten Mal, wenn eine Rechnung für denselben Kunden erstellt werden soll, genügt ein Klick auf *Fill Template* und die Auswahl des gewünschten Templates. Das Eingabeformular öffnet sich mit allen gespeicherten Werten vorausgefüllt und der Nutzer muss lediglich die sich ändernden Felder wie *datum*, *r.-nr.* und *preis* aktualisieren und kann dann direkt auf *Generate PDF* klicken.
Dieser Workflow reduziert nicht nur den Eingabeaufwand für wiederkehrende Dokumente erheblich sondern auch die Fehleranfälligkeit: Statt neun Felder komplett neu auszufüllen, werden nur zwei bis drei situationsabhängige Felder geändert.

6 Fazit und Ausblick

Die vorangegangenen Kapitel haben gezeigt, wie ein hybrides System aus klassischer Bildverarbeitung und Large Language Models aufgebaut werden kann, um Formularfelder in PDF-Dokumenten automatisch zu erkennen und zu klassifizieren. Kapitel 2 legte die Systemgrundlage, Kapitel 3 entwickelte den regelbasierten Algorithmus, Kapitel 4 ergänzte ihn durch LLM-Unterstützung und Kapitel 5 bettete alles in eine nutzbare Webanwendung ein. Dieses abschließende Kapitel bewertet die Ergebnisse der Arbeit. Abschnitt 6.1 beantwortet die zentrale Forschungsfrage. Abschnitt 6.2 diskutiert Stärken und Schwächen des gewählten Ansatzes. Abschnitt 6.3 reflektiert die getroffenen technischen Designentscheidungen und Abschnitt 6.4 zeigt mögliche Erweiterungen und Verbesserungen für die Zukunft auf.

6.1 Beantwortung der Forschungsfrage

Die zentrale Forschungsfrage lautete, wie Computer-Vision-Algorithmen zur automatischen Erkennung und Klassifizierung von Eingabefeldern in PDF-Formularen eingesetzt werden können und welche technischen Herausforderungen dabei bestehen.
Die Arbeit zeigt, dass eine rein regelbasierte Bildverarbeitung für einfache Formulare durchaus praktikabel ist. Der in Kapitel 3 beschriebene Algorithmus nutzt Kantendetektion nach Canny[1] und Konturerkennung nach Suzuki und Abe[2], um Formularfelder geometrisch zu lokalisieren. Die Label-Feld-Zuordnung erfolgt anschließend über räumliche Näheberechnung. Für Standardformulare funktioniert dieser Ansatz zuverlässig und ohne externe Abhängigkeiten. Bei komplexeren Layouts wie unregelmäßigen Formularstrukturen, Checkboxen oder unkonventionell positionierten Labels stößt der klassische Ansatz jedoch an seine Grenzen. Auf dem Auftragsformular (Kapitel 4.1.1) ordnete der traditionelle Algorithmus 6 von 9 Feldern korrekt zu (67 %), während Gemini 2.5 Flash alle 9 Felder richtig erkannte (100 %). Auf dem Bewerbungsformular (Kapitel 4.1.2) erreichte der traditionelle Algorithmus 9 von 14 Feldern (64 %), das LLM 12 von 14 (86 %). Die verbleibenden zwei Felder waren Linienfelder, die von keinem der beiden Ansätze erkannt werden konnten, da OpenCV keine geschlossene Kontur findet.
Die technischen Herausforderungen lagen dabei vor allem in drei Bereichen: der korrekten Koordinatentransformation zwischen PDF- und Bildkoordinatensystem, der zuverlässigen Konturdetektion bei schlecht gedruckten oder gescannten Dokumenten sowie der konsistenten JSON-Ausgabe der Sprachmodelle unter verschiedenen Eingabebedingungen. Alle drei Probleme wurden im Rahmen der Arbeit adressiert. Die Koordinatentransformation durch einen expliziten Umrechnungsschritt, die Konturdetektion durch gezielte Bildvorverarbeitung, und die JSON-Validierung durch einen Retry-Mechanismus im `LabelFieldAgent`.
Insgesamt lässt sich die Forschungsfrage damit positiv beantworten: Der gewählte hybride Ansatz aus regelbasierter Bildverarbeitung und optionaler LLM-Unterstützung eignet sich für die automatische Formularfelderkennung und bietet einen praktikablen Kompromiss zwischen Aufwand, Kosten und Erkennungsqualität. Die entwickelte Lösung erfüllt

[1] Vgl. Canny, 1986, S. 679.
[2] Vgl. Suzuki/Abe, 1985, S. 32.

damit ihren Anspruch als Proof-of-Concept und legt eine technische Grundlage, auf der weiterführende Arbeiten aufbauen können.

6.2 Stärken und Schwächen des Ansatzes

6.2.1 Stärken

Die wichtigste Stärke des Systems ist die vollständige Automatisierung der Formularfelderkennung. Ohne das entwickelte System müssten Feldpositionen, Feldnamen und Koordinaten manuell in eine Konfiguration eingetragen werden. Das dauert lange und ist fehleranfällig, besonders bei Formularen mit vielen Feldern. Das System übernimmt diesen Schritt automatisch: Ein PDF wird eingelesen, die Felder werden erkannt, benannt und direkt als wiederverwendbares Template gespeichert.
Eine weitere Stärke ist die Robustheit des traditionellen Algorithmus bei einfachen Formularen. Wenn keine Anomalien auftreten, erkennt der Algorithmus zuverlässig alle Eingabefelder und die dazugehörigen Label.
Darüber hinaus deckt das System den gesamten Workflow in einer einzigen Anwendung ab: von der automatischen Felderkennung über die Verwaltung von Templates und Datensätzen bis zur finalen Ausgabe einer ausgefüllten PDF-Kopie. Dieser durchgängige Ansatz vermeidet Medienbrüche und macht das System direkt nutzbar, ohne dass mehrere separate Tools kombiniert werden müssen.
Das Dataset-System stellt einen zusätzlichen praktischen Mehrwert dar, der über die eigentliche Forschungsfrage hinausgeht. Durch gespeicherte Datensätze mit Alias-Unterstützung können Nutzer häufig auszufüllende Dokumente schneller erstellen, da bereits bekannte Werte nicht erneut eingegeben werden müssen. Im beschriebenen Beispieldurchlauf reduziert sich der manuelle Eingabeaufwand von neun Einzelfeldern auf wenige Klicks.

6.2.2 Schwächen

Die größte Schwäche des Systems liegt in der Abhängigkeit von der Qualität der Eingabedokumente. Der Computer-Vision-Algorithmus setzt voraus, dass Formularfelder als Rechtecke mit erkennbaren Kanten vorliegen. Formulare ohne sichtbare Feldumrandungen, etwa Felder die als Linien statt als Boxen dargestellt sind, werden vom aktuellen Algorithmus nicht erkannt, wie das Bewerbungsformular in Kapitel 4.1.2 zeigt. Hier müsste die Erkennungsphase grundlegend erweitert werden.
Auch die LLM-Anbindung hat Einschränkungen. Der Nicht-Determinismus der Modelle bedeutet, dass dasselbe Dokument bei wiederholter Verarbeitung unterschiedliche Ergebnisse liefern kann. Die Abhängigkeit von externen APIs birgt Datenschutzrisiken, da Dokumenteninhalte an externe Server übertragen werden. Für sensible Dokumente ist Ollama als lokale Alternative vorhanden, aber die Qualität lokaler Modelle ist deutlich geringer als die der cloudbasierten Dienste.
Die Skalierbarkeit bei mehrseitigen Dokumenten ist ein weiteres offenes Problem. Aktuell wird jede Seite separat verarbeitet, und der Token-Verbrauch steigt linear mit der Seitenanzahl. Bei einem geschätzten Verbrauch von ca. 1090 Tokens pro Seite (vgl. Kapitel 4) und Kosten von etwa $0.001 pro Seite für Gemini ist das für Einzeldokumente akzeptabel, könnte bei großen Batch-Verarbeitungen aber relevant werden.
Schließlich ist das System als Einzelnutzer-Prototyp ausgelegt. Es fehlt ein vollständiges Authentifizierungssystem mit Nutzerkonten und Zugriffsrechten, das für einen Mehrbenutzerbetrieb notwendig wäre.

6.3 Technische Designentscheidungen und ihre Bewertung

Im Laufe der Entwicklung wurden mehrere Designentscheidungen getroffen, die für einen Prototyp sinnvoll waren, bei einer Weiterentwicklung aber überdacht werden sollten.

Regelbasierter statt lernbasierter Ansatz für die Felderkennung: Da es bei der Entwicklung um ein Proof-of-Concept ging, ist ein regelbasierter Ansatz für den Computer Vision Algorithmus besser nachzuvollziehen. Regelbasierte Ansätze sind deterministisch und benötigen keine Trainingsdaten. Das ist ein klarer Vorteil für einen Prototyp, bei dem zunächst die grundsätzliche Machbarkeit gezeigt werden soll. Der Nachteil zeigt sich bei komplexeren Layouts: Da der Algorithmus ausschließlich auf geometrischen Merkmalen basiert (Rechteckform, Kantenstärke, räumliche Zusammenhänge), versagt er bei Formularen, die von diesen Annahmen abweichen. Ein trainiertes Modell wie LayoutLM[3] würde solche Abweichungen durch gelernte Muster besser tolerieren, hätte aber einen erheblichen Mehraufwand bei der Datenbeschaffung und dem Training erfordert. Für den Zweck dieser Arbeit war der regelbasierte Ansatz die richtige Wahl. Für ein Produktivsystem wäre ein hybrider Ansatz mit einem vortrainierten Dokumentenmodell denkbar.

SQLite als Datenbank: Die Wahl von SQLite als dateibasiertes Datenbanksystem war für den Prototypbetrieb gut geeignet. Keine separate Serverinstanz, einfache Einrichtung, keine Konfiguration. Für einen Produktiveinsatz mit mehreren gleichzeitigen Nutzern wäre eine Migration zu PostgreSQL sinnvoll. Django unterstützt diesen Wechsel ohne Codeänderungen, lediglich die Datenbankverbindung in `settings.py` müsste angepasst werden.

JSONField für variable Feldstrukturen: Die Feldwerte von Generations und Datasets werden als JSON in der Datenbank gespeichert, anstatt in normalisierten Tabellen. Das ermöglicht variable Feldstrukturen ohne Schemaänderungen, ein klarer Vorteil während der Entwicklungsphase, in der sich die Feldstruktur häufig änderte. Der Nachteil ist, dass SQL-Abfragen auf einzelne Feldwerte nicht ohne weiteres möglich sind. Für spätere Auswertungen (z. B. Statistiken über häufig verwendete Werte) wäre ein normalisiertes Schema eventuell besser geeignet.

Erweiterbarkeit: Die LLM-Anbindung ist über den `LabelFieldAgent` abstrahiert, sodass neue Provider ohne Änderungen an der restlichen Codebasis ergänzt werden können. Das Alias-System des Dataset-Konzepts lässt sich auf weitere Anwendungsfälle ausdehnen. Das Django-Authentifizierungssystem ist vorhanden und müsste für einen Mehrbenutzerbetrieb nur konfiguriert werden.

6.4 Potenzielle Verbesserungen und zukünftige Erweiterungen

Aus den identifizierten Schwächen ergeben sich mehrere konkrete Ansätze für eine Weiterentwicklung.

Verbesserte Felderkennung: Der aktuelle Algorithmus erkennt nur rechteckige, umrandete Felder. Eine sinnvolle Erweiterung wäre die Unterstützung von linienbasierten Feldern (Unterstriche statt Boxen), die in vielen traditionellen Formularen verbreitet sind. Dazu könnten Liniendetektion über die Hough-Transformation sowie ein erweitertes Regelwerk für die Zuordnung von Labels zu Linien implementiert werden.

Batch-Verarbeitung: Aktuell können nur einzelne PDFs verarbeitet werden. Für Anwendungsfälle, in denen viele gleichartige Formulare auf einmal verarbeitet werden müssen,

[3] Vgl. Xu et al., 2020, S. 1192.

wäre eine Batch-Upload-Funktion sinnvoll.

Genauere Leistungsmessung: Die in Kapitel 4 präsentierten Ergebnisse basieren auf einer begrenzten Anzahl von Testdokumenten. Für eine belastbarere Bewertung wäre ein standardisiertes Testset mit verschiedenen Formulartypen (einfache Formulare, mehrspaltige Layouts, gemischte Formate) und eine systematische Auswertung über mehrere Durchläufe notwendig. Das würde auch die Variabilität durch den LLM-Nicht-Determinismus quantifizierbar machen.

Mehrbenutzerbetrieb: Die Integration des bestehenden Django-Authentifizierungs- systems würde es ermöglichen, Templates und Datasets benutzerspezifisch zu verwalten. Jeder Nutzer würde nur seine eigenen Templates sehen, was das System für den Einsatz in Unternehmen mit mehreren Mitarbeitern geeignet machen würde.

Insgesamt stellt die vorliegende Arbeit einen funktionsfähigen Prototyp dar, der die grundsätzliche Machbarkeit des hybriden Ansatzes aus Computer Vision und LLM-Unterstützung für die Formularfelderkennung belegt. Die erzielten Ergebnisse zeigen, dass der Ansatz nicht nur technisch realisierbar, sondern auch praktisch einsetzbar ist. Die Architektur ist so gestaltet, dass die genannten Erweiterungen ohne grundlegende Umstrukturierung umgesetzt werden können.

Literaturverzeichnis

[Canny, 1986] Canny, J. (1986). A computational approach to edge detection. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, PAMI-8(6):679–698.

[Django Software Foundation, 2024] Django Software Foundation (2024). Django documentation. Abgerufen am 26. März 2026.

[Douglas and Peucker, 1973] Douglas, D. H. and Peucker, T. K. (1973). Algorithms for the reduction of the number of points required to represent a digitized line or its caricature. *Cartographica: The International Journal for Geographic Information and Geovisualization*, 10(2):383–384.

[International Telecommunication Union, 2011] International Telecommunication Union (2011). Studio encoding parameters of digital television for standard 4:3 and wide-screen 16:9 aspect ratios. Technical Report BT.601-7, ITU-R.

[Sahoo et al., 2024] Sahoo, P., Singh, A. K., Saha, S., Jain, V., Mondal, S., and Chadha, A. (2024). A systematic survey of prompt engineering in large language models: Techniques and applications. *arXiv preprint arXiv:2402.07927*.

[Smith, 2007] Smith, R. W. (2007). An overview of the tesseract OCR engine. In *Ninth International Conference on Document Analysis and Recognition (ICDAR 2007)*. IEEE.

[Sobel and Feldman, 1968] Sobel, I. and Feldman, G. (1968). A 3x3 isotropic gradient operator for image processing. In *Stanford Artificial Intelligence Project (SAIL)*.

[Suzuki and Abe, 1985] Suzuki, S. and Abe, K. (1985). Topological structural analysis of digitized binary images by border following. *Computer Vision, Graphics, and Image Processing*, 30:32–46.

[Xu et al., 2020] Xu, Y., Li, M., Cui, L., Huang, S., Wei, F., and Zhou, M. (2020). Layoutlm: Pre-training of text and layout for document image understanding. In *Proceedings of the 26th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining*, pages 1192–1200. ACM.

Anhang

Anhang 1. Quellcode

Der vollständige Quellcode ist verfügbar unter:
`https://github.com/bensalo/openPDF`

Anhang 2. Erklärung

Die Erklärung ist verfügbar unter:
`https://github.com/bensalo/openPDF/blob/main/bs_erklaerung.pdf`

Erklärung

Ich versichere hiermit, dass ich die vorliegende Arbeit selbständig verfasst und keine anderen als die im Literaturverzeichnis angegebenen Quellen und Hilfsmittel benutzt habe.

Stellen, die wörtlich oder sinngemäß aus veröffentlichten oder noch nicht veröffentlichten Quellen entnommen sind, sind als solche kenntlich gemacht. Zudem sind die Stellen gekennzeichnet, die mit Hilfsmitteln erstellt wurden.

Die Zeichnungen oder Abbildungen in dieser Arbeit sind von mir selbst erstellt worden oder mit einem entsprechenden Quellen oder Hilfsmittelnachweis versehen.

Diese Arbeit ist in gleicher oder ähnlicher Form noch bei keiner anderen Prüfungsbehörde eingereicht worden.

Aachen, im Monat Jahr
April 2026

Ben Salomon
(Vollständige, handschriftliche Unterschrift)